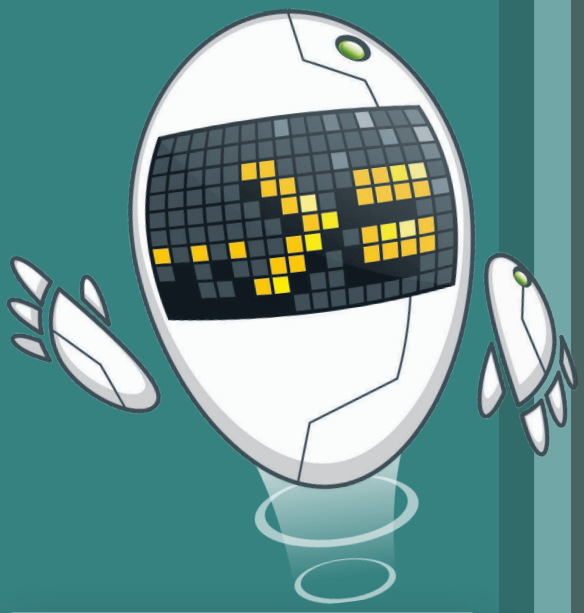
الوحدة الثالثة:
البرمجة مع بايثون

ستتعلم في هذه الوحدة كيفية استخدام بيئة التواصل باي تشارم (PyCharm Community) من أجل إنشاء مقاطع برمجية تعالج مشاكل أكثر صعوبة في لغة بايثون (Python). وستتعلم أيضًا كيفية اتخاذ القرار عند البرمجة باستخدام معاملات شرطية ومنطقية جنبًا إلى جنب مع العبارات في بايثون. بالإضافة إلى ذلك ستتعرف على الحلقات (loops) وكيفية استخدامها في بايثون، وستستخدم الحلقات المتداخلة (nested loops) من أجل طباعة الأنماط، كما ستتعلم كيفية تجميع مجموعة من التعليمات البرمجية وإنشاء الدوال. أخيرًا، ستتعلم كيفية التعامل مع ملف إكسل باستخدام بايثون.

أهداف التعلم

ستتعلم في هذه الوحدة:

- كيفية إنشاء مقطع برمجي في بيئة التواصل باي تشارم (PyCharm Community).
- كيفية استخدام الأنواع المختلفة من الجمل الشرطية.
- التمييز بين الجمل الشرطية المختلفة.
- استخدام الحلقات في بايثون.
- إيقاف الحلقة باستخدام عبارة الإيقاف.
- آلية استخدام الحلقات المتداخلة.
- إنشاء أنماط باستخدام الحلقات.
- ماهية الدالة في لغة البرمجة.
- بناء دوال جديدة .
- التعامل مع معاملات الدالة.
- تمييز المتغيرات المحلية والعامة.
- استخدام التعليمات البرمجية لمعالجة جداول البيانات.



الأدوات

- إصدار بيئة التواصل باي تشارم (PyCharm Community Edition)

وزارة التعليم
Ministry of Education
2025 - 1446

هل تذكر؟

مراحل إنشاء البرنامج



أولًا: تحديد المشكلة.



ثانيًا: كتابة الخطوات الخوارزمية اللازمة لحلها بترتيب متسلسل.



ثالثًا: رسم المخطط الانسيابي بناءً على الخطوات الخوارزمية.



رابعًا: كتابة المقطع البرمجي بلغة البرمجة.

الأشكال المستخدمة في المخطط الانسيابي

الوصف	نوع الصندوق
الإشارة إلى بداية ونهاية المقطع البرمجي.	البداية / النهاية
استقبال وعرض البيانات التي سيتم معالجتها (إدخال وإخراج).	الإدخال / الإخراج
تنفيذ عملية رياضية.	العمليات
اتخاذ قرارات (نعم أو لا) أو اختبارات تحقق (صواب أو خطأ).	اتخاذ قرار
عرض التسلسل الذي يجب تنفيذ الخطوات به.	↓

وزارة التعليم
Ministry of Education
2026 - 1448

أمثلة على أنواع البيانات

نوع البيانات	التعريف في بايثون	مثال
الأعداد الصحيحة	int	900000, 0, 999, 12
الأعداد الحقيقية	float	3.0, -90.5, 0.0003, 4.5
النصوص والرموز	str	"$$$", "hello", "Saad"
قيمة منطقية	bool	True, False

أنت تطلب من المستخدم إدخال قيمة للمتغير x.

```
print("الرجاء إدخال قيمة للمتغير x: ")
x=input()
print("قيمة x:",x)
```

يُدخل المستخدم القيمة 10 ويضغط على Enter ↵.

```
الرجاء إدخال قيمة للمتغير x:
10
قيمة x: 10
```

تم إسناد القيمة 10 للمتغير x.

العمليات الرياضية في لغة البايثون	
في بايثون	رياضيًا
2 + 4	2 + 4
2 - 4	2 - 4
2 * 4	2x4
2 / 4	2 ÷ 4
x ** 2	x^2

وزارة التعليم
Ministry of Education
2026 - 1448

المعاملات (operators) في بايثون

المعامل في لغة البرمجة هو رمز يستخدم لإجراء عملية محددة على المتغيرات والقيم.

يقدم بايثون فئات مختلفة من المعاملات. الفئات الأربع للمعاملات الأكثر استخدامًا في بايثون هي:

> المعاملات الرياضية.

> معاملات الإسناد.

> المعاملات الشرطية.

> المعاملات المنطقية.

المعاملات (operators) في بايثون:		
المعاملات الرياضية	+ - * / **	تُستخدم لإجراء العمليات الحسابية: الجمع، الطرح، الضرب، و القسمة وما إلى ذلك.
معاملات الإسناد	= += -= *= /=	تُستخدم لإسناد قيم للمتغيرات.
المعاملات الشرطية	> < == <= >= !=	تُستخدم في مقارنة القيم أثناء كتابة الجمل الشرطية.
المعاملات المنطقية	and or not	تُستخدم لفحص أكثر من شرط في جملة شرطية واحدة، وأيضًا لفحص نقيض الشرط. وتُمكّن هذه المعاملات من اتخاذ قرارات لجمل شرطية مركبة.

رابط الدرس الرقمي

www.ien.edu.sa

الدرس الأول:

المعاملات الشرطية والمعاملات المنطقية في بايثون

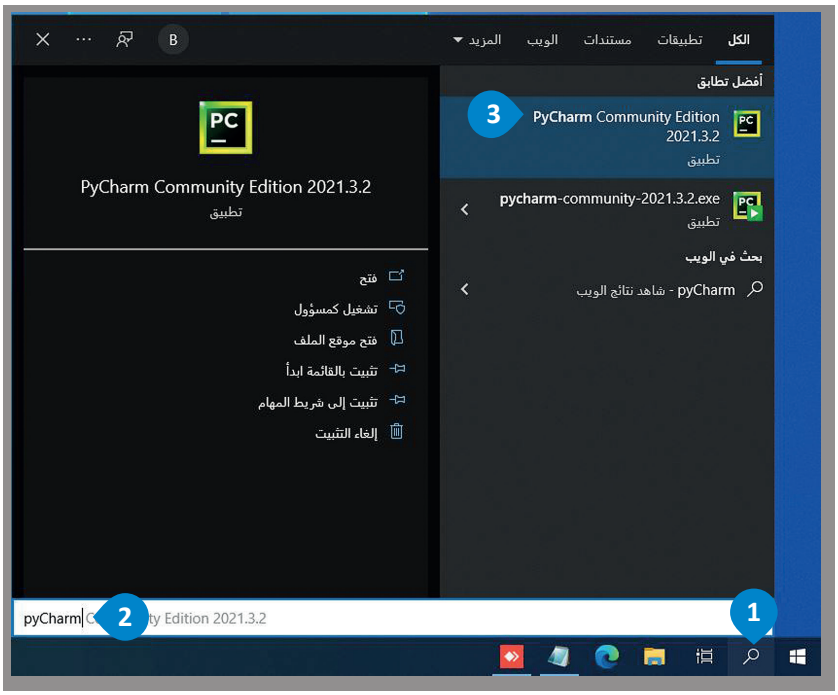
بعد أن تعرفت على لغة برمجة **بايثون** (Python)، حان الوقت لتتعرف على بيئة برمجية جديدة لكتابة مقاطعك البرمجية وهي **بيئة التواصل باي تشارم** (PyCharm Community).

بيئة التواصل باي تشارم

بيئة التواصل باي تشارم تُستخدم خصيصًا للغة برمجة بايثون. وهي بيئة تطوير متكاملة (Integrated Development Environment - IDE) توفر العديد من الوحدات النمطية والحزم والأدوات لمساعدتك في البرمجة بلغة بايثون. تتوافق بيئة التواصل باي تشارم مع أنظمة تشغيل **ويندوز** (Windows) و**لينكس** (Linux) و**ماك أو إس** (macOS).

يمكنك تنزيل بيئة التواصل باي تشارم مجانًا من الموقع الإلكتروني:

https://www.jetbrains.com/pycharm/



لفتح بيئة التواصل باي تشارم :

> اضغط على زر **Search** (بحث). 1

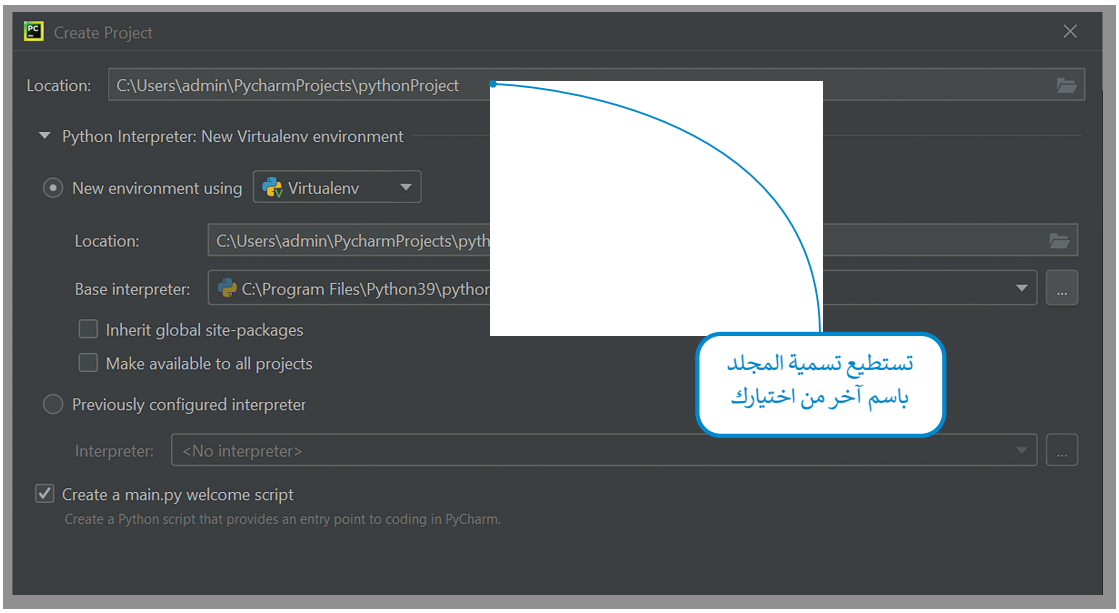
> اكتب **PyCharm** (باي تشارم) في شريط البحث، 2 ثم اضغط عليه. 3

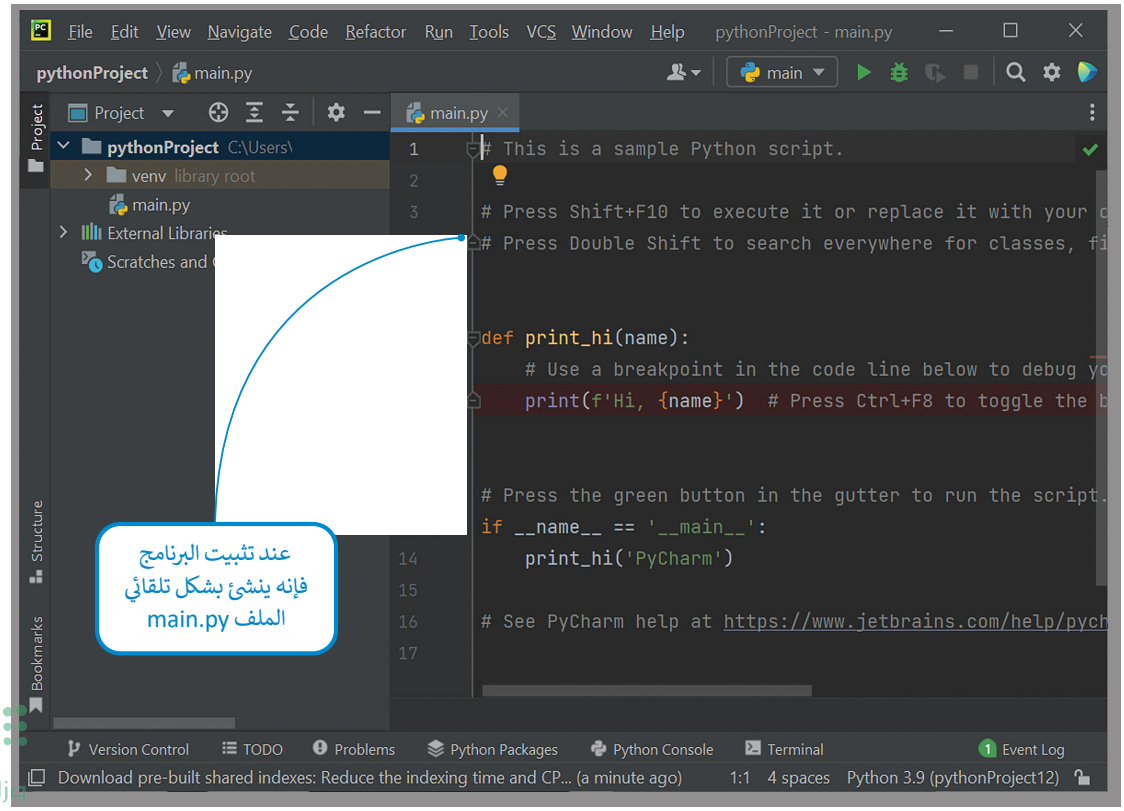
يمكنك استخدام منصات عبر الإنترنت تدعم لغة بايثون، مثل جوجل كولاب (Google Colab)، لكتابة وتشغيل البرامج دون الحاجة إلى تثبيت بيئة برمجية على جهاز الحاسب الخاص بك.

معلومة

على الرغم من أن **بيئة التطوير المتكاملة** (Integrated Development Environment - IDE) تساعد المطور على كتابة المقاطع البرمجية بسهولة إلا أنها أيضًا توفر وظائف وأدوات أكثر تعقيدًا.

من أجل الحفاظ على ملفاتك منظمة، تحتاج إلى مجلد لحفظها. في المرة الأولى التي تقوم فيها بتثبيت بيئة التواصل باي تشارم على جهاز الحاسب الخاص بك، ينشئ البرنامج تلقائيًا مجلد باسم **PythonProject**، وتُحفظ الملفات التي تقوم بإنشائها في هذا المجلد. وبهذه الطريقة يمكنك الوصول بسهولة إلى جميع ملفات بايثون الخاصة بك.





وزارة التعليم
Ministry of Education
2026 - 1448

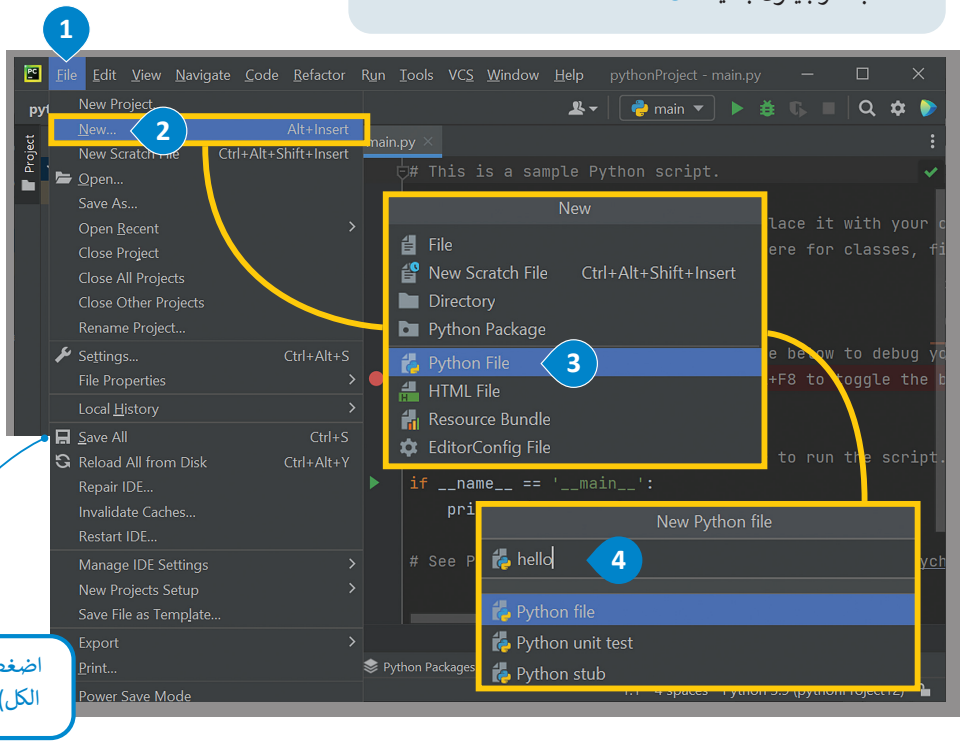
إنشاء ملف بايثون في بيئة التواصل باي تشارم وتشغيله

حان الوقت الآن لإنشاء أول ملف لك بلغة بايثون في بيئة التواصل باي تشارم.

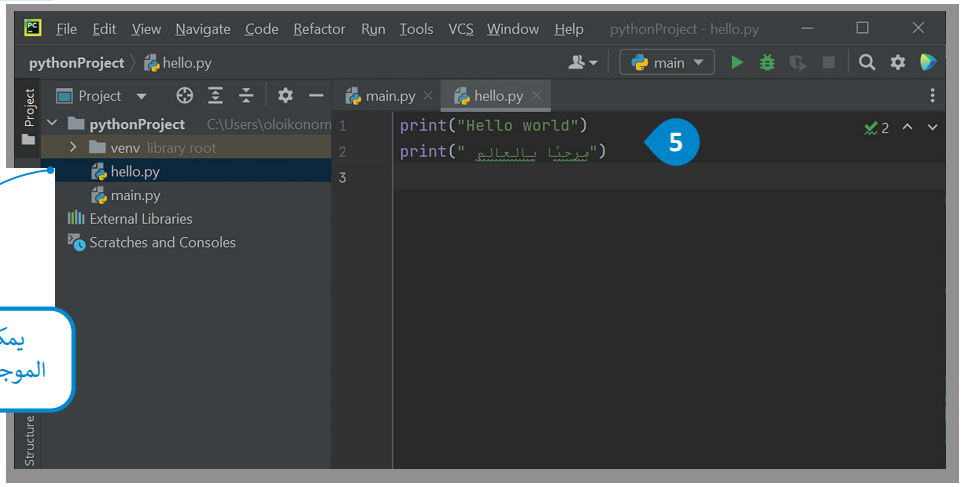
لإنشاء ملف جديد:

- اضغط على **File** (ملف)، 1 وحدّد **New** (جديد). 2
- حدد ملف **Python File** (ملف بايثون). 3
- اكتب اسمًا لملفك. 4 واضغط على زر **Enter**.
- اكتب أمر بايثون بسيط. 5

يحفظ باي تشارم بشكل تلقائي التغييرات التي تجريها على ملفاتك.



اضغط على **Save All** (حفظ الكل) لحفظ التغييرات يدويًا.



يمكنك هنا رؤية الملفات الموجودة في مجلد مشروعك.

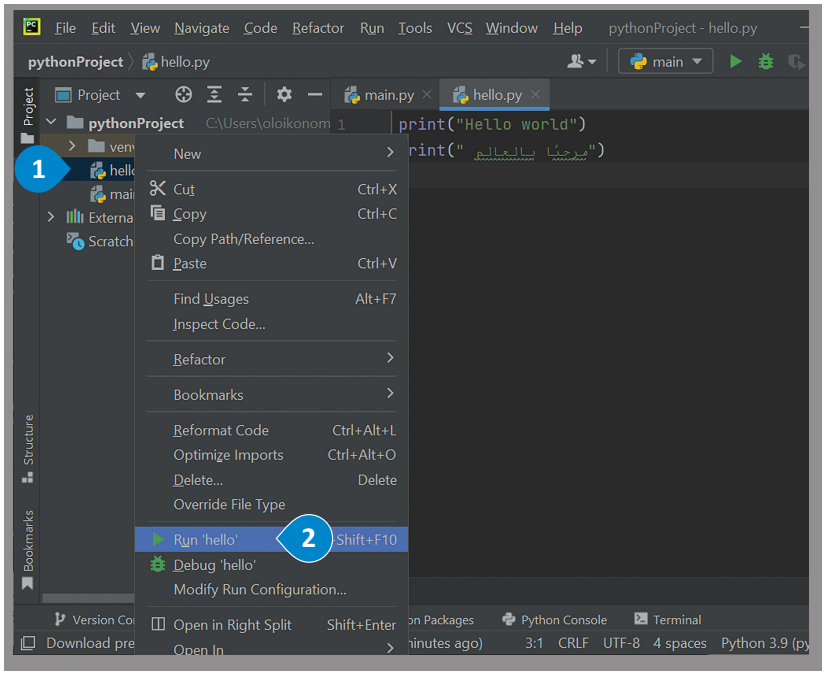
أنت الآن جاهز لتشغيل مقطعك البرمجي الأول في بيئة التواصل باي تشارم .

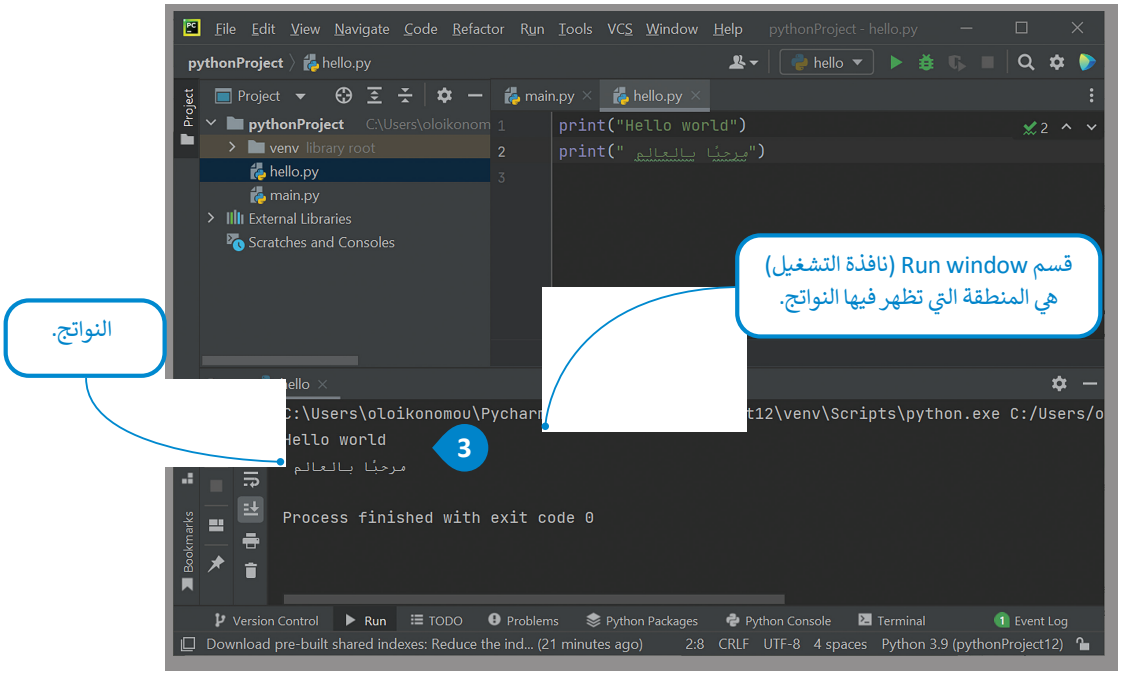
لتشغيل المقطع البرمجي الخاص بك:

> اضغط بزر الفأرة الأيمن على ملفك. 1

> اختر **Run 'file name'** (تشغيل 'اسم الملف'). 2

> تتم معالجة المقطع البرمجي وتظهر النتيجة أسفل الشاشة. 3





معلومة

أثناء عملية المعالجة يتم فحص وكشف الأخطاء في المقطع البرمجي.

المعاملات الشرطية في بايثون

لاتخاذ قرار في البرمجة، عليك استخدام المعاملات الشرطية. تُستخدم هذه المعاملات للتحكم في مسار المقطع البرمجي. ستجدها عادة كجزء من دالة **if** أو **while**، حيث تقارن القيم وتعود بنتيجة **صواب** (True) أو **خطأ** (False). وبناءً على نتيجة الفحص، ينفذ المقطع البرمجي العمليات المقابلة.

المعامل	المعنى
==	يساوي
>	أكبر من
<	أصغر من

المعامل	المعنى
>=	أكبر من أو يساوي
<=	أصغر من أو يساوي
!=	لا يساوي

فيما يلي تتم مقارنة قيمتين ويعود المقطع البرمجي بلغة بايثون بالإجابة المنطقية: صواب (True) أو خطأ (False).

مثال 1: المعاملات الشرطية

```
x=5
y=6
z=x==y
print(z)
```

False

```
x=5
y=6
k=x<y
print(k)
```

True

```
x=5
y=6
n=x+y!=15
print(n)
```

True

```
x=5
y=5
m=x-y<=0
print(m)
```

True

معلومة

من الأخطاء الشائعة، الخلط بين معامل الإسناد (=) ومعامل المساواة (==). وللتفريق بينهما تجد أن معامل الإسناد سيغير قيمة المتغير، في حين أن معامل المساواة سيقرأ فقط قيمة المتغير ويرى ما إذا كان يساوي شيئًا ما.

وزارة التعليم
Ministry of Education
2026 - 1448



الناتج

```
a=5>7
print(a)
```

```
b=8<=8
print(b)
```

```
c=5+9>=14
print(c)
```

المعاملات المنطقية في بايثون

قد تحتاج إلى دمج مقارنات متعددة في المعاملات الشرطية، في هذه الحالة تُستخدم المعاملات المنطقية، وكذلك تُستخدم لدمج شرطين أو أكثر.

من أجل فهم استخدام المعاملات المنطقية بشكل أفضل، يمكنك الرجوع إلى جدول الحقيقة.

جدول الحقيقة هو جدول يسرد جميع المدخلات الممكنة للمتغير، ويعرض مخرجاته وفقًا للمعامل المنطقي كما يظهر بالأسفل:

المعامل	المعنى
AND	تعود القيمة صواب (True) إذا كانت كلتا العبارتين صحيحة.
OR	تعود القيمة صواب (True) إذا كانت إحدى العبارات صحيحة.
NOT	تعود القيمة العكسية خطأ إذا كان الناتج صواب، وصواب إذا كان الناتج خطأ.

ترتيب المعاملات في البرمجة
( )
**
* /
+ -
== > < <= >= =!
not
and
or

جدول الحقيقة

A	B	A and B	A or B	not A	not B
True	True	True	True	False	False
False	True	False	True	True	False
True	False	False	True	False	True
False	False	False	False	True	True

وزارة التعليم
Ministry of Education
2026 - 1448

مثال 2: المعاملات المنطقية

في الأمثلة التالية سترى كيفية استخدام العوامل المنطقية في المقاطع البرمجية.

```
x=5
y=6
k=x<10 and y<8
print(k)
```

True

```
x=True
y=False
z=x==y
print(z)
```

False

```
x=True
y=False
m=(x or y) and (not False)
print(m)
```

True

```
x=5
y=6
n=x>y and (not y==6)
print(n)
```

False



الناتج

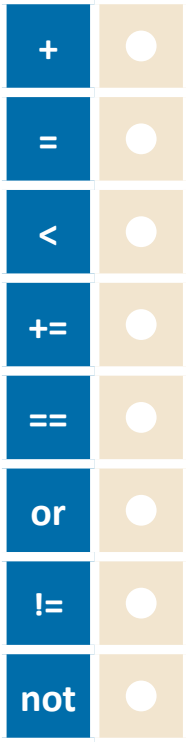
```
a=True
b=False
c=False
z=(a or b) and (not c)
print(z)
```

```
a=True
b=True
c=True
z=(a or b) and (not c)
print(z)
```

لنطبق معًا

تدريب 1

صل كل معامل مع فئته.



1. المعاملات الرياضية
2. معاملات الإسناد
3. المعاملات الشرطية
4. المعاملات المنطقية

تدريب 2

اكتب المعامل الصحيح بجانب الوصف الخاص به.

1. يساوي	
2. أكبر من	
3. أصغر من أو يساوي	
4. لا يساوي	

تدريب 3

اكتب ناتج العمليات التالية:

a=2

b=5

c=10

```
x = a>b and b>c   <--------
y = (a+b)<c or (b+c)<a   <--------
z = not(a>0 or b<0)   <--------
```

تدريب 4

استخدم بيئة التواصل باي تشارم لكتابة مقطع برمجي يقوم بقراءة رقمين ويحسب حاصل الجمع وكذلك حاصل الطرح وحاصل الضرب لهما ثم اكتبه:

- المقطع البرمجي لإيجاد حاصل جمع الرقمين
- المقطع البرمجي لإيجاد حاصل طرح الرقمين
- المقطع البرمجي لإيجاد حاصل ضرب الرقمين

Num1= _____

Num2= _____

اكتب الأوامر البرمجية هنا.

اكتب الأمر البرمجي الذي يجمع الرقمين ويظهر الناتج <--------

اكتب الأمر البرمجي الذي يطرح الرقمين ويظهر الناتج <--------

اكتب الأمر البرمجي الذي يضرب الرقمين ويظهر الناتج <--------

رابط الدرس الرقمي

www.ien.edu.sa

الدرس الثاني:

الجمل الشرطية في البايثون

تعرفت على تنفيذ الأوامر بالتتابع واحدًا تلو الآخر، إلاّ أنّ اتخاذ القرار أحيانًا يكون تبعًا لطبيعة المشكلة. إنّ اتخاذ القرار عملية مهمة وقوية متاحة لكل مبرمج يقوم بتطوير البرنامج. ستتعلم في هذا الدرس كيفية إنشاء مقاطع برمجية تستجيب لمدخلات من المستخدم أثناء تنفيذها وتعطي نتائج مختلفة تبعًا للمدخلات المختلفة. ويتحقق ذلك باستخدام الجُمل الشرطية.

الجُمل الشرطية في بايثون

لاتخاذ القرارات في لغة برمجة بايثون: استخدم الجملة الشرطية **if**. ويوجد ثلاثة أنواع من الجمل الشرطية كما في الشكل التالي:

أنواع الجُمل الشرطية

جملة if الشرطية البسيطة

```
if الشرط:
    عبارة 1
```

جملة if...else الشرطية

```
if الشرط:
    عبارة 1
else:
    عبارة 2
```

جملة if...elif

```
if الشرط:
    عبارة 1
elif:
    عبارة 2
else:
    عبارة 3
```



أمثلة على أنواع الجمل الشرطية

حان الوقت لتشاهد مثالًا.

```
grade=17
if grade>=10:
    print("ناجح")
```

ناجح

if الشرط:
العبارة

```
grade=7
if grade>=10:
    print("ناجح")
else:
    print("تحتاج إلى المحاولة أكثر")
```

تحتاج إلى المحاولة أكثر

if الشرط:
عبارة 1
else:
عبارة 2

```
grade=12
if grade>=15:
    print("ممتاز")
elif grade>=10:
    print("جيد جدًا")
else:
    print("تحتاج إلى المحاولة أكثر")
```

جيد جدًا

if الشرط:
عبارة 1
elif:
عبارة 2
else:
عبارة 3

if الشرط:
العبارة

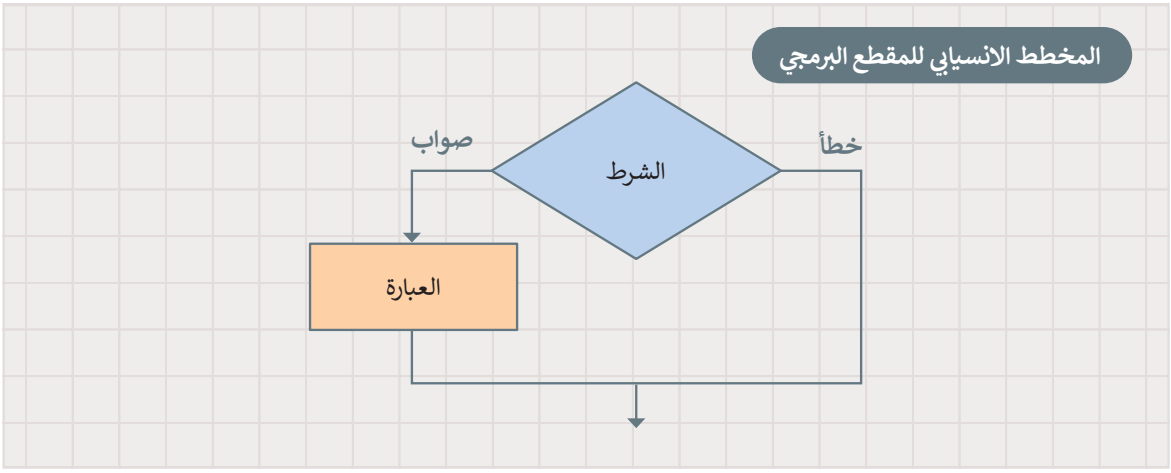
لاحظ أن النقطتين (:) اللتين تليان التعبير الشرطي ضروريتان.

وزارة التعليم
Ministry of Education
2026 - 1448

جملة if الشرطية البسيطة

if الشرط:
    العبارة

> إذا تحقق الشرط فستنفّذ العبارة أو العبارات التي تتبع **if**.
> إذا لم يتحقق الشرط فلن تنفّذ العبارة أو العبارات التي تتبع **if**.



المسافة البادئة

المسافة البادئة (Indentation) في لغة البرمجة بايثون مهمة جدًا، إذ يستخدمها المقطع البرمجي للإشارة إلى العبارات المعتمدة على تحقيق الشرط. إذا لم تترك مسافة بادئة، فستتلقى رسالة خطأ.

الصفوف غير المزودة بمسافة بادئة والتي لا يعتمد تنفيذها على نتيجة الشرط، ستنفّذ حتى إذا لم يتحقق الشرط.

```
grade=int(input("اكتب الدرجة"))
if grade>=10:
print("ناجح")
```

```
IndentationError: expected an indented block
```

```
grade=18
if grade>=15:
    print("ممتاز")
print("استمر في تطوير مهاراتك")
```

لا ينتمي أمر الطباعة الثاني إلى جملة If لذلك يُنفذ الأمر دون النظر إلى نتيجة جملة if.

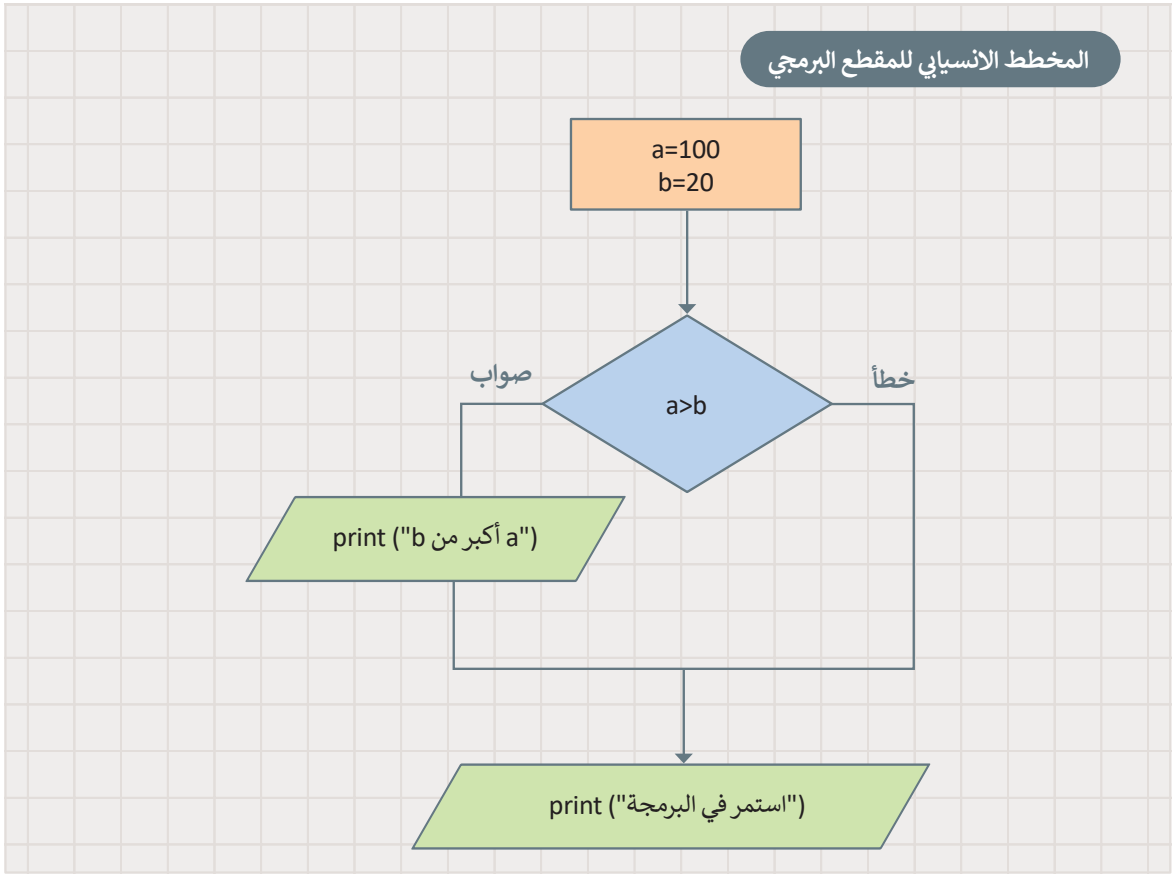
```
ممتاز
استمر في تطوير مهاراتك
```

وزارة التعليم
Ministry of Education
2026 - 1448

حان الوقت لتشاهد بعض الأمثلة.

مثال 1: مقارنة رقمين

يقارن المقطع البرمجي قيمة الرقمين ثم يطبع أيهما أكبر.




```
a = 100
b = 20
if a > b:
    print("a أكبر من b")
print ("استمر في البرمجة")
```

a أكبر من b
استمر في البرمجة

الشرط

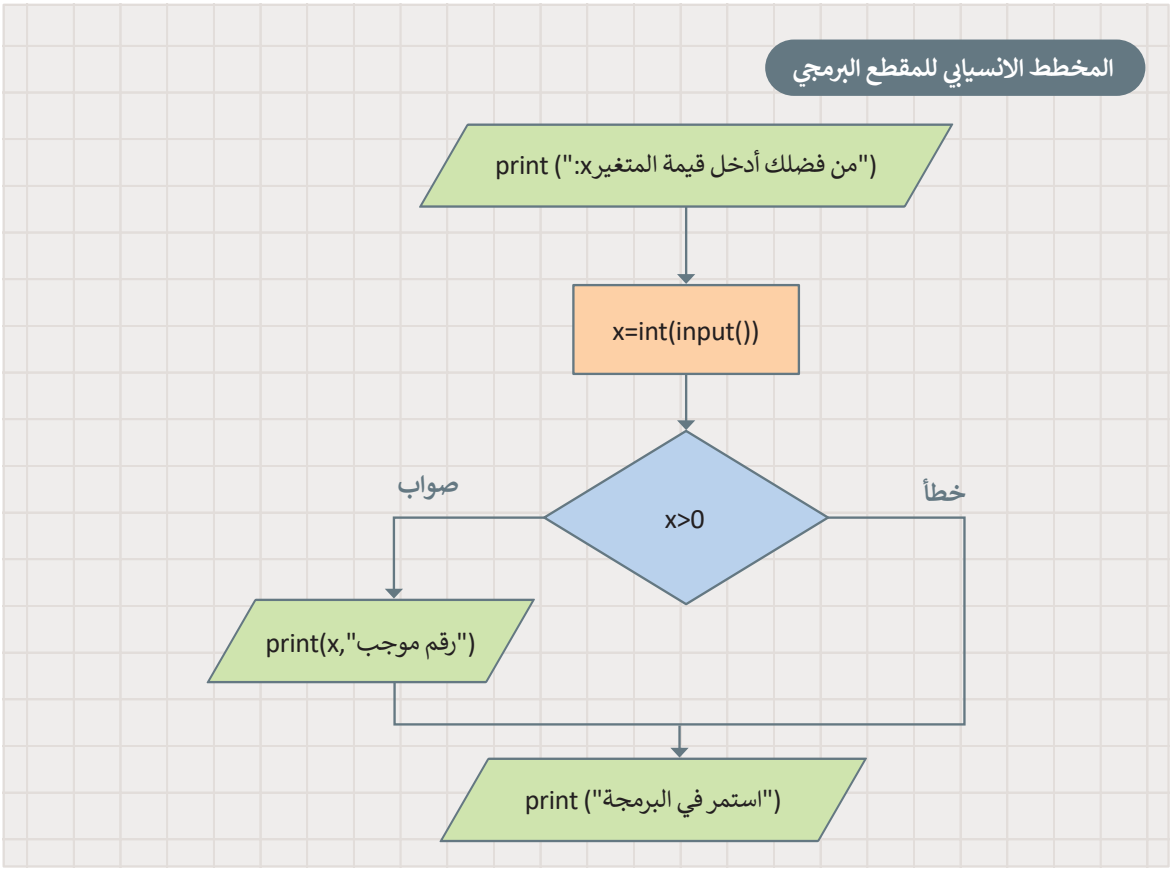
العبارة المعتمدة على تحقق الشرط

وزارة التعليم
Ministry of Education
2026 - 1448

مثال آخر على جملة if البسيطة.

مثال 2: رقم موجب

يقرأ المقطع البرمجي رقمًا ويتحقق مما إذا كان الرقم موجبًا.




```
print("من فضلك أدخل قيمة المتغير:x")
x=int(input())
if x>0:
    print(x,"رقم موجب")
print ("استمر في البرمجة")
```

```
من فضلك أدخل قيمة المتغير:x
5
5 رقم موجب
استمر في البرمجة
```

لنطبق معًا

تدريب 1

ماذا يحدث عند تشغيل المقطع البرمجي التالي:

اختر الإجابة الصحيحة		

.1
`Number = 12`
`if Number > 0 :`
`  print("رقم موجب")`
	لن يعمل المقطع البرمجي لوجود خطأ في صيغة الأوامر.	○

	لن تُعرض أي رسالة على الشاشة لأن الشرط لم يتحقق.	○
	ستعرض الرسالة " رقم موجب" على الشاشة.	○

.2
`Number != 12`
`if Number > 0 :`
`  print("رقم موجب")`
	لن يعمل المقطع البرمجي لوجود خطأ في صيغة الأوامر.	○

	لن تُعرض أي رسالة على الشاشة لأن الشرط لم يتحقق.	○
	ستعرض الرسالة " رقم موجب" على الشاشة.	○

تدريب 2

اكتب ناتج المقطع البرمجي التالي:

```
a = -7
if a < 0 :
    a = a*(-1)
print(a)
```

الناتج

وزارة التعليم
Ministry of Education
2026 - 1448

تدريب 3

اكتب ناتج المقطع البرمجي التالي:
مستخدمًا القيم: a=5, b=12, c=9

```
print("من فضلك أدخل قيمة a و b و c:")
a=int(input())
b=int(input())
c=int(input())
if a > b and a > c :
    m = a
if b > a and b > c :
    m = b
if c > a and c > b :
    m =c
print(m)
```

ما وظيفة المقطع البرمجي؟

تدريب 4

ارسم المخطط الانسيابي لمقطع برمجي يتحقق من كون عمرك مناسبًا لقيادة السيارة، ثم اكتب المقطع البرمجي.

المخطط الانسيابي للمقطع البرمجي

المقطع البرمجي

رابط الدرس الرقمي

www.ien.edu.sa

الدرس الثالث:

اتخاذ القرارات

بعد أن تعلمت كيفية كتابة الجمل الشرطية البسيطة في لغة البرمجة بايثون، وتعرفت على ما سيحدث إذا تحقق الشرط، حان الوقت لتتعلم ما سيحدث عندما لا يتحقق الشرط.

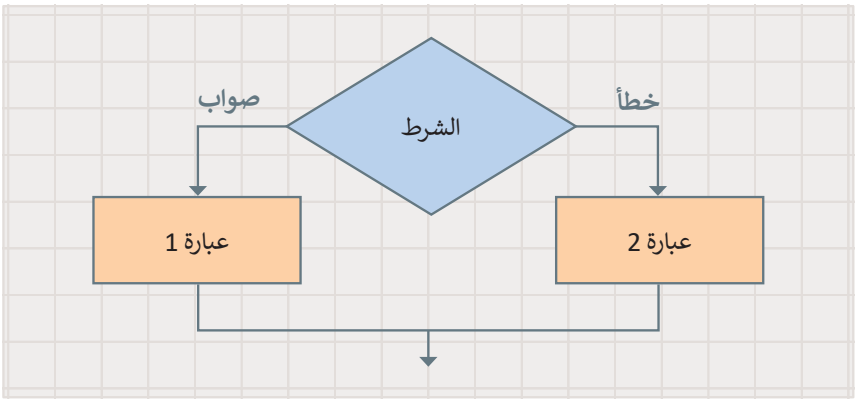
جملة if...else الشرطية

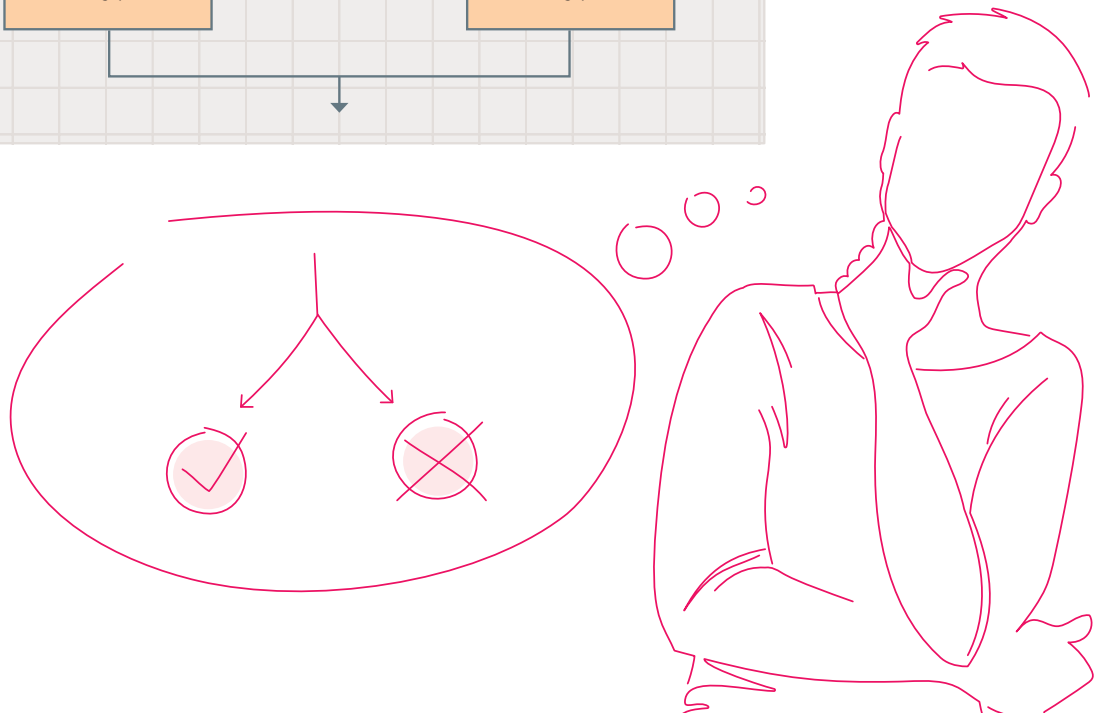
ستتعلم في هذا الدرس كيفية استخدام جملة **if ... else** الشرطية.

إذا تحقق الشرط فسيتم تنفيذ عملية أو عمليات محددة، وإذا لم يتحقق فسيتم تنفيذ عملية أو عمليات أخرى محددة.

كما في الحالة السابقة، يتم استخدام المسافة البادئة لتحديد العبارات التي ستنفّذها كل مرة.

```
if الشرط:
    عبارة 1
else:
    عبارة 2
```



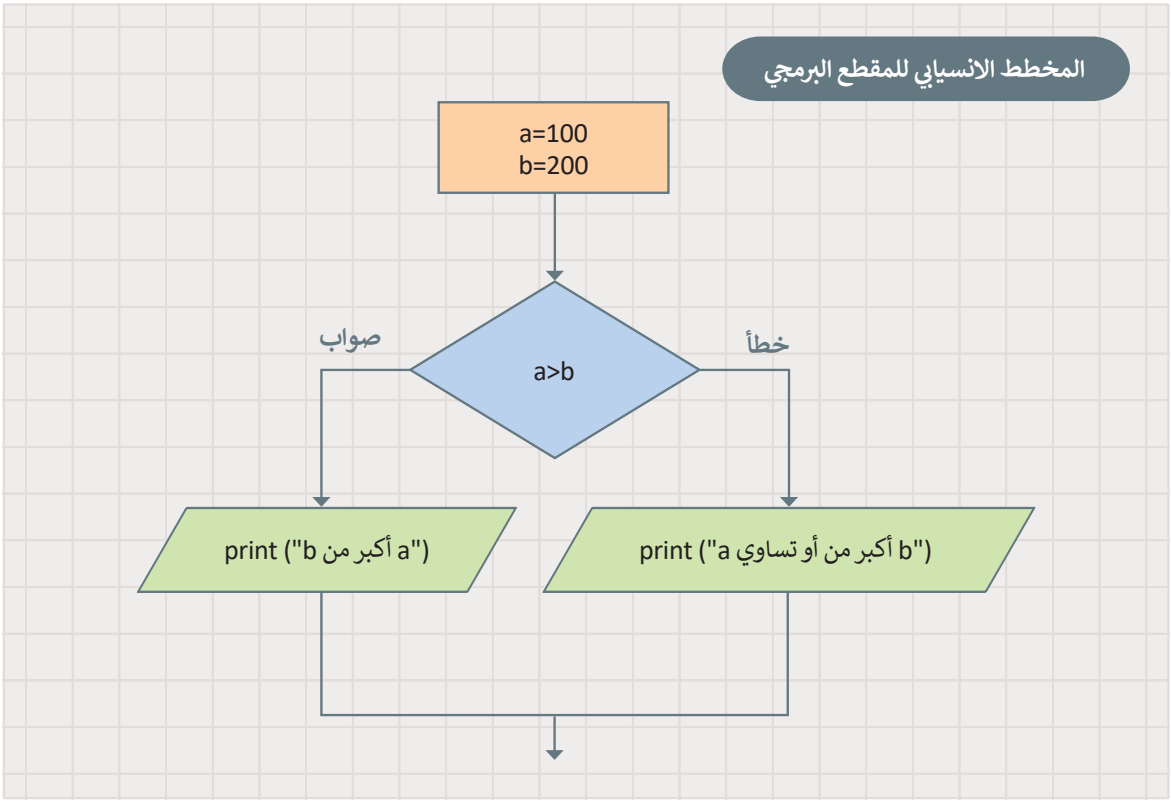


وزارة التعليم
Ministry of Education
2026 - 1448

حان الوقت لتشاهد بعض الأمثلة.

مثال 1: مقارنة رقمين

يقارن المقطع البرمجي قيمة الرقمين ثم يطبع أيهما أكبر.




```
a = 100
b = 200
if a > b:
    print("a أكبر من b")
else:
    print("b أكبر من أو تساوي a")
```

الشرط

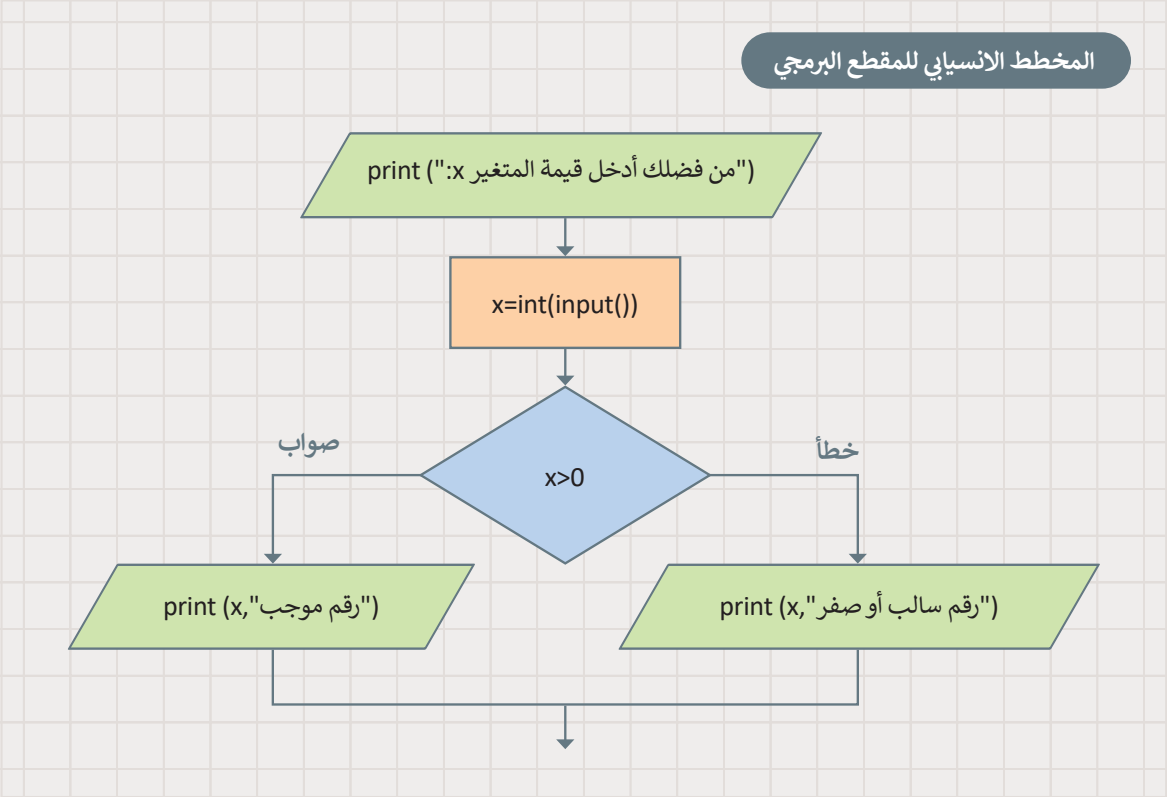
عبارة 1

عبارة 2

b أكبر من أو تساوي a

مثال 2: موجب أو سالب

يقرأ المقطع البرمجي الرقم ويتحقق مما إذا كان الرقم موجبًا أم سالبًا ثم يطبع الرسالة المقابلة.



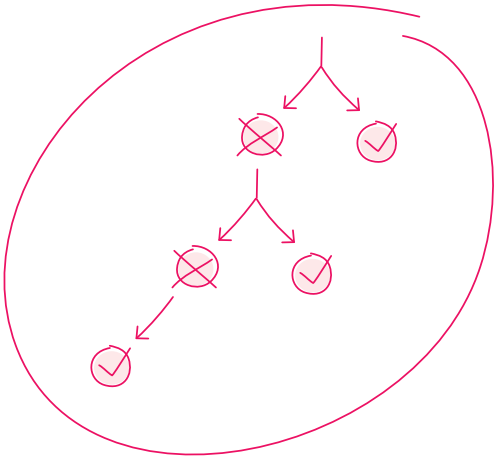

```
print("من فضلك أدخل قيمة المتغير x:")
x=int(input())
if x>0:
    print(x,"رقم موجب")
else:
    print(x,"رقم سالب أو صفر")
```

```
من فضلك أدخل قيمة المتغير x:
-2
-2 رقم سالب أو صفر
```

جملة if...elif

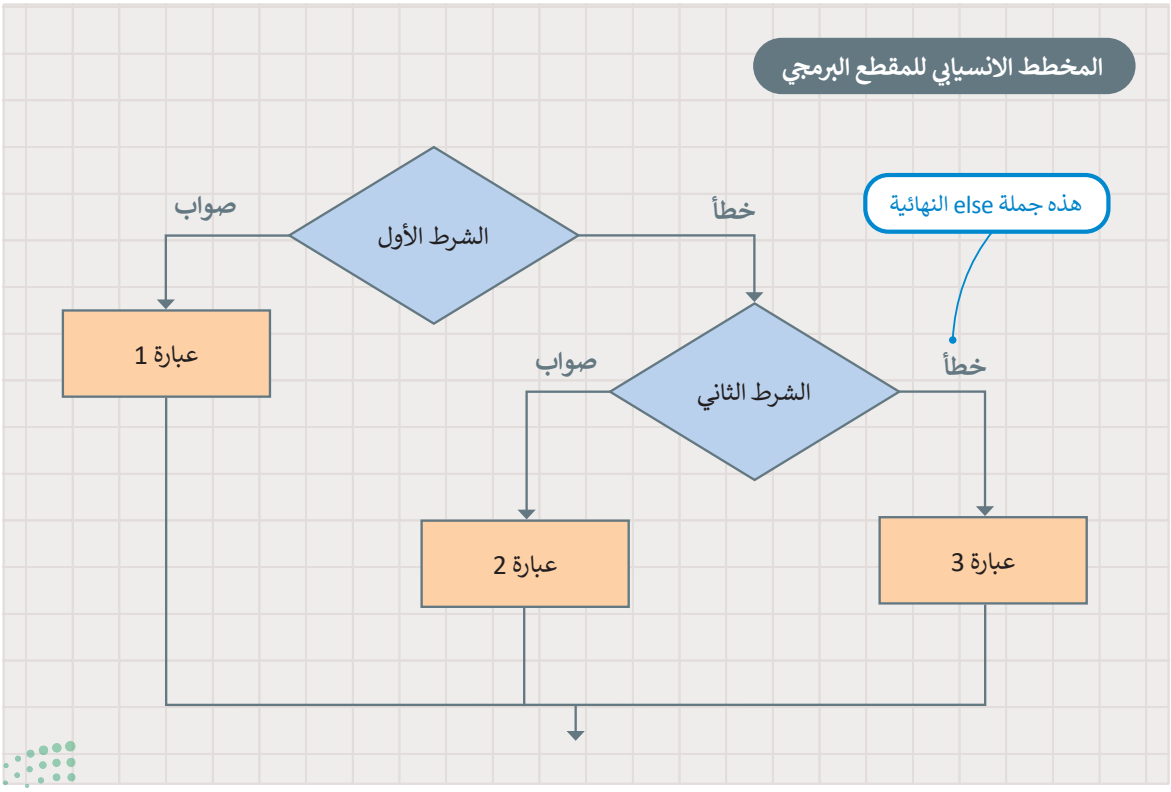
في الجمل الشرطية السابقة كان على المستخدم اختيار أحد خيارين، أما في هذا النوع من جمل **if** الشرطية، فإن المستخدم يجب أن يحدّد خيارًا من بين خيارات متعددة، تنفّذ عبارات **if** من الأعلى إلى الأسفل.

يتحقق المقطع البرمجي من الشروط واحدًا تلو الآخر، فإذا تحقق أحد الشروط، يتم تنفيذ ما تحت هذا الشرط ويتجاوز باقي الشروط، أما إذا لم يتحقق أيًّا من الشروط، فستنفّذ جملة **else**.



```
if الشرط الأول:
    عبارة 1
elif الشرط الثاني:
    عبارة 2
else:
    عبارة 3
```

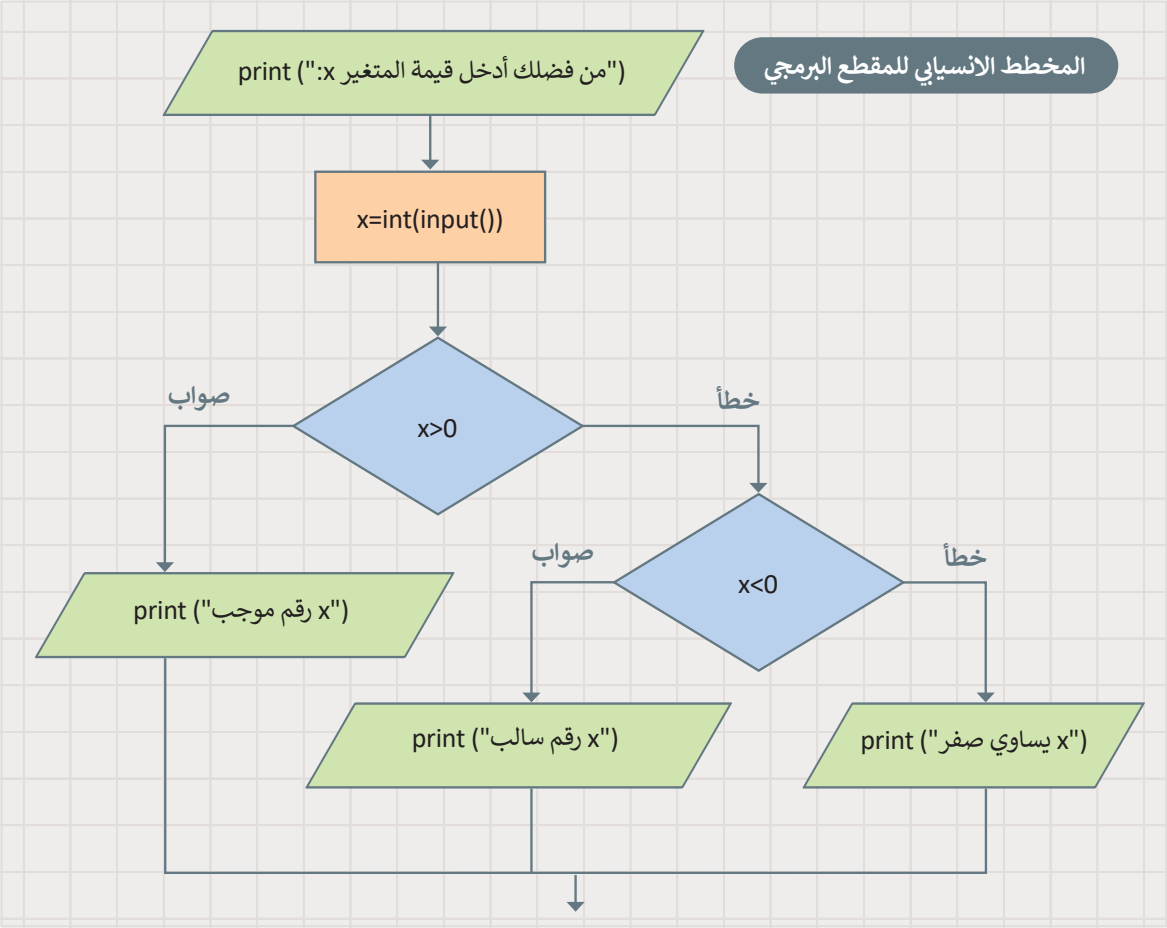
المخطط الانسيابي للمقطع البرمجي



حان الوقت لتشاهد بعض الأمثلة.

مثال 1: موجب أو سالب أو صفر

يستخدم المقطع البرمجي جملة **if ... elif**. ليقرأ رقمًا ويتحقق مما إذا كان الرقم موجبًا أم سالبًا أم صفرًا ثم يطبع الرسالة المقابلة.




```
print("من فضلك أدخل قيمة المتغير x:")
x=int(input())
if x>0:
    print("رقم موجب x")
elif x<0:
    print("رقم سالب x")
else:
    print("يساوي صفر x")
```

من فضلك أدخل قيمة المتغير x:
-45
رقم سالب x

وزارة التعليم
Ministry of Education
2026 - 1448

مثال 2: درجات الطلبة

يقرأ المقطع البرمجي درجة الطالب ثم يطبع الرسالة المقابلة.

```
print("من فضلك أدخل الدرجة:")
g=int(input())
if g<0 or g>10:
    print("درجة غير صالحة")
elif g>=8:
    print("ممتاز")
elif g>=5:
    print("جيد جدًا")
else:
    print("اجتهد أكثر")
```

```
من فضلك أدخل الدرجة:
12
درجة غير صالحة
```



الدرجة	
ما الذي يجب عليك إدخاله حتى يتم طباعة "جيد جدًا"؟	
ما الذي يجب عليك إدخاله حتى يتم طباعة "اجتهد أكثر"؟	
ما الذي يجب عليك إدخاله حتى يتم طباعة "ممتاز"؟	

لنطبق معًا

تدريب 1

ارسم المخطط الانسيابي للمقطع البرمجي.

```
print("من فضلك أدخل الدرجة:")
g=int(input())
if g<0 or g>10:
    print("درجة غير صالحة")
elif g>=8:
    print("ممتاز")
elif g>=5:
    print("جيد جدًا")
else:
    print("اجتهد أكثر")
```

المخطط الانسيابي للمقطع البرمجي

وزارة التعليم
Ministry of Education
2026 - 1448

تدريب 2

وفقًا للمقطع البرمجي الذي أمامك:

ما نتيجة المتغير (num) إذا كانت:

a: num = 18

b: num = -7

```
num=int(input(":أدخل رقم"))
if num>=0:
    print(num)
else:
    num=num*(-1)
    print(num)
```

a

b

ما وظيفة المقطع البرمجي؟

ارسم المخطط الانسيابي للمقطع البرمجي.

المخطط الانسيابي للمقطع البرمجي

تدريب 3

ارسم مخططًا انسيابيًا لإدخال درجة حرارة اليوم ويطبع إحدى الجُمل التالية:

- "طقس معتدل"، اذا كانت درجة الحرارة محصورة بين 15 و 30.
- "طقس بارد"، اذا كانت درجة الحرارة أقل من 15.
- "طقس حار"، إذا كانت درجة الحرارة أكبر من 30.

ثم اكتب المقطع البرمجي.

المخطط الانسيابي للمقطع البرمجي

المقطع البرمجي

وزارة التعليم
Ministry of Education
2026 - 1448

رابط الدرس الرقمي

www.ien.edu.sa

الدرس الرابع:

الشروط المتداخلة

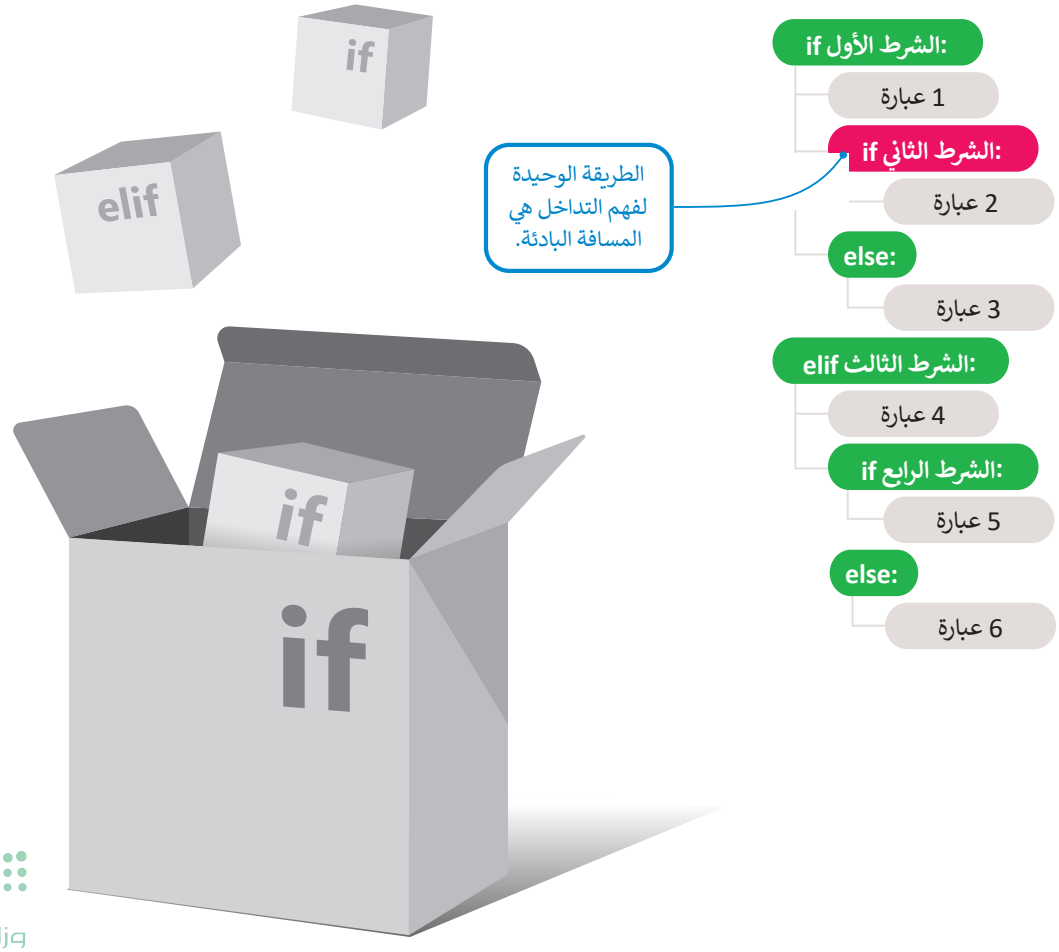
التداخل

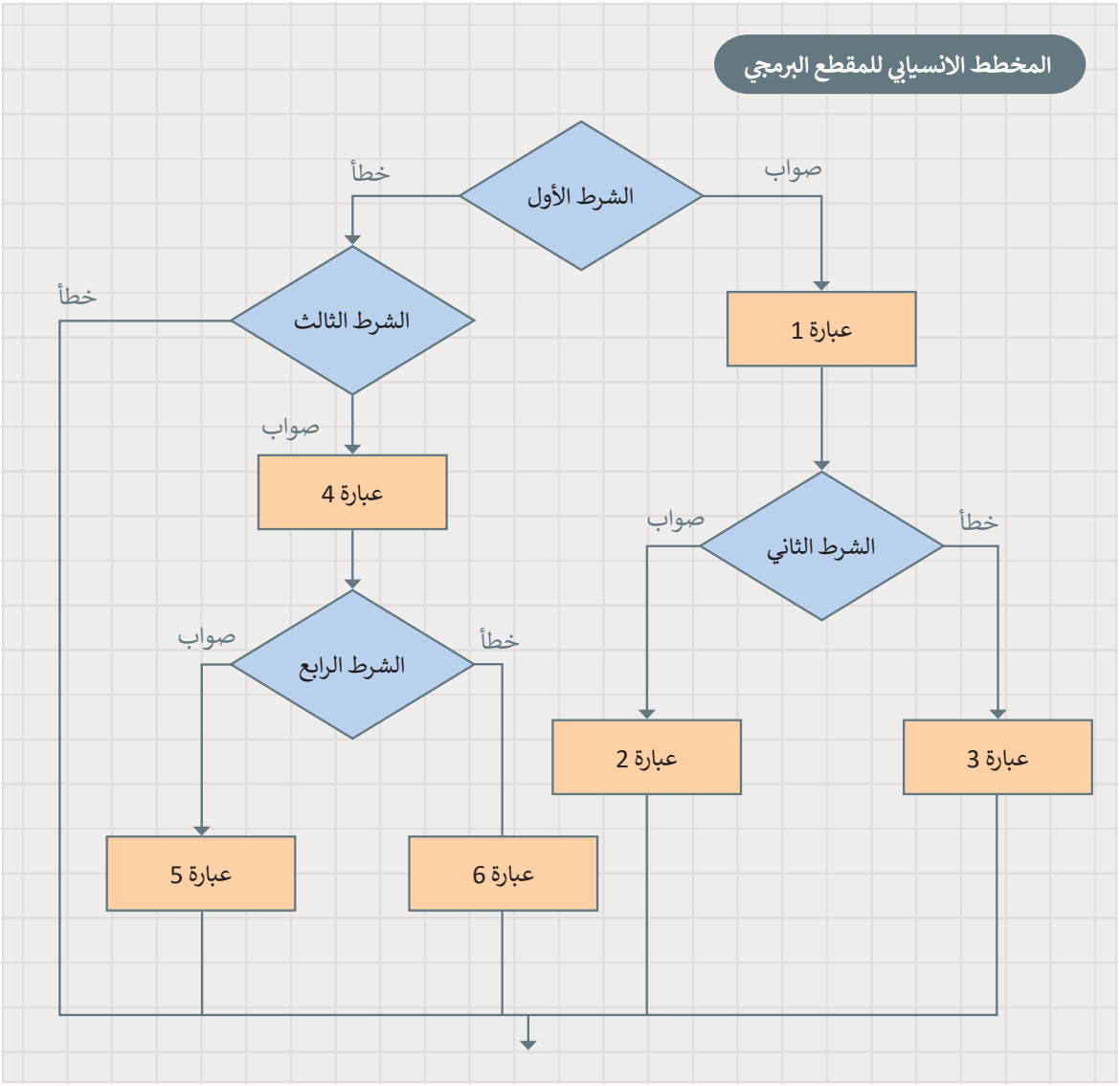
التداخل (Nesting) مصطلح يستخدم لوصف وضع كائن أو أكثر داخل كائن آخر. في برمجة جهاز الحاسب، العبارة المتداخلة عبارة موجودة داخل عبارة أخرى في المقطع البرمجي الأساسي للبرنامج. عند استخدام العبارات المتداخلة تُستخدم المسافة البادئة لتحديد الشرط الذي تنتمي إليه العبارة.

الجملة الشرطية if المتداخلة

الجملة الشرطية **if** المتداخلة عبارة عن جملة **if** البسيطة وتكون موجودة داخل (أو متداخلة مع) جملة **if** الأخرى أو جملة **if...else** الشرطية.

يمكن دمج أي عدد من العبارات في أي مجموعة داخل بعضها البعض.

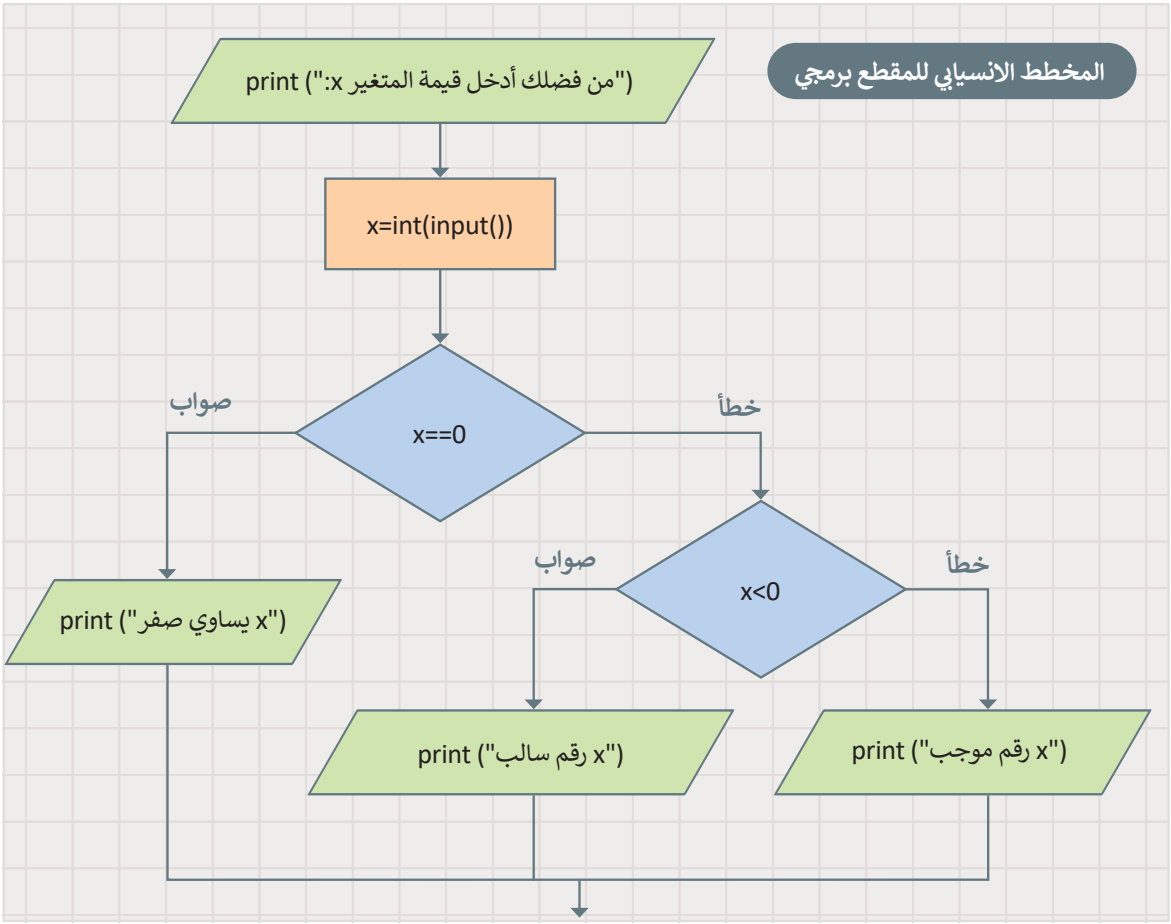



المخطط الانسيابي للمقطع البرمجي
الشرط الأول
خطأ
صواب
الشرط الثالث
خطأ
عبارة 1
صواب
عبارة 4
الشرط الثاني
صواب
خطأ
الشرط الرابع
صواب
خطأ
عبارة 2
عبارة 3
عبارة 5
عبارة 6

شاهد بعض الأمثلة السابقة باستخدام الشروط المتداخلة.

مثال 1: جملة if المتداخلة

يستخدم المقطع البرمجي عبارة **if** المتداخلة لطباعة الرقم إذا كان الرقم المدخل موجبًا أو سالبًا أو صفرًا.



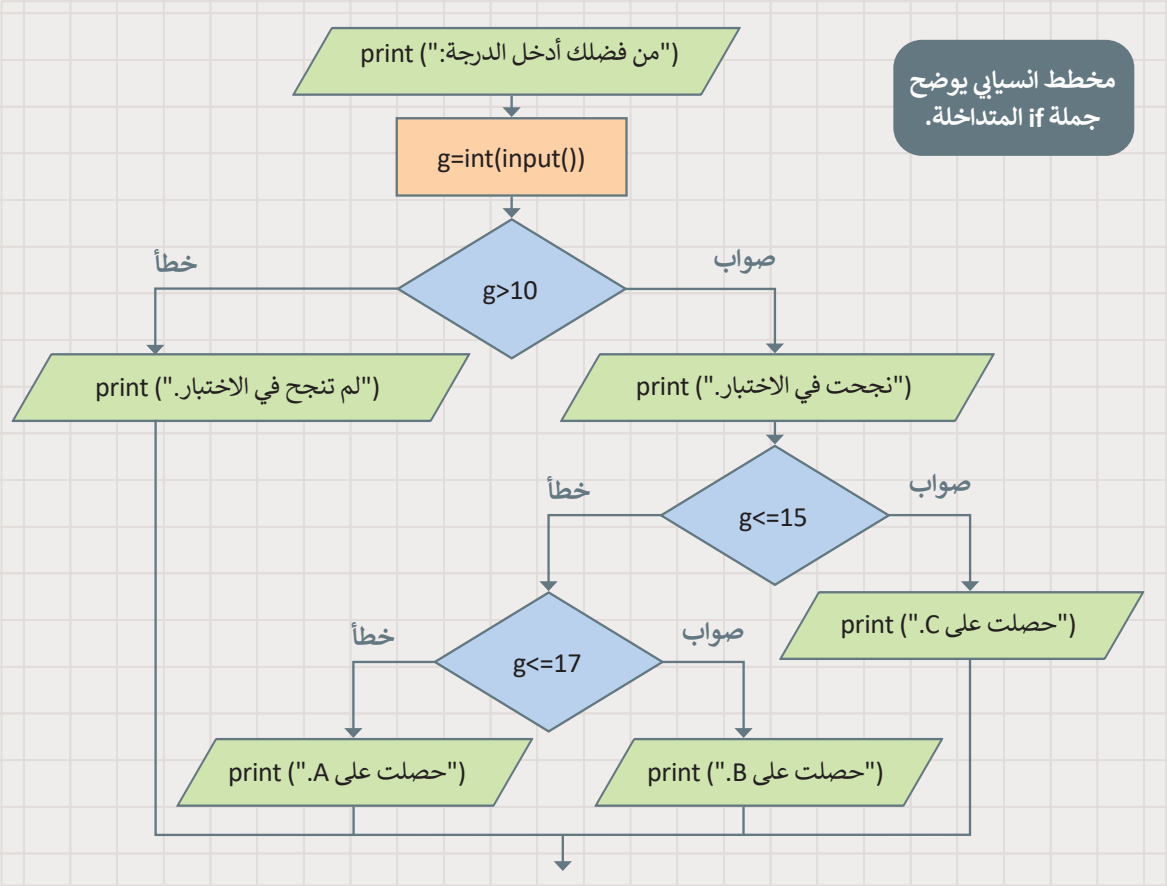

```
print("من فضلك أدخل قيمة المتغير :x")
x=int(input())
if x==0:
    print("x يساوي صفر")
else:
    if x<0:
        print("x رقم سالب")
    else:
        print("x رقم موجب")
```

```
من فضلك أدخل قيمة المتغير :x
7
x رقم موجب
```

وزارة التعليم
Ministry of Education
2026 - 1448

مثال 2: تقديرات الطلبة بالأحرف

يحسب المقطع البرمجي تقدير الطالب بالأحرف. لترى كيف يمكنك استخدام جملة **if** المتداخلة لإخبار الطالب إذا نجح في الاختبار والتقدير الذي حصل عليه بالأحرف.




```
print("من فضلك أدخل الدرجة:")
g=int(input())
if g>10:
    print("نجحت في الاختبار.")
    if g<=15:
        print("حصلت على C.")
    elif g<=17:
        print("حصلت على B.")
    else:
        print("حصلت على A.")
else:
    print("لم تنجح في الاختبار.")
```

```
من فضلك أدخل الدرجة:
16
نجحت في الاختبار.
حصلت على B.
```


وزارة التعليم
Ministry of Education
2026 - 1448

لنطبق معًا

تدريب 1

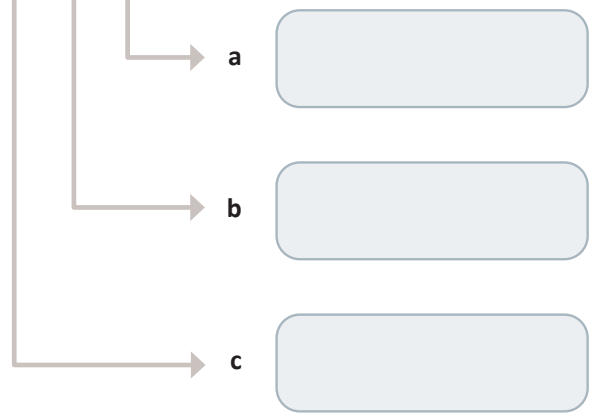
ماذا سيعرض المقطع البرمجي على الشاشة إذا أدخلت قيمة الشهر (month) كالتالي:

```
month=int(input("اكتب رقم الشهر: "))
if month<1 or month>13:
    print("رقم الشهر خاطئ")
else:
    if month>=9 and month<=11:
        print("فصل الخريف")
    elif month==12 or month>=1 and month<=2:
        print("فصل الشتاء")
    elif month>=3 and month<=5 :
        print("فصل الربيع")
    else:
        print("فصل الصيف")
```

a 4

b 1

c 25



تدريب 2

يمكن للاعب أن ينضم لفريق كرة السلة إذا كان طوله أكبر من 1.80 متر، ووزنه بين 85 و125 كجم.

> ارسم المخطط الانسيابي لمقطع برمجي يقرأ طول ووزن اللاعب الرياضي، ويعرض إذا كان بإمكانه الانضمام إلى فريق كرة السلة ثم اكتب المقطع البرمجي.

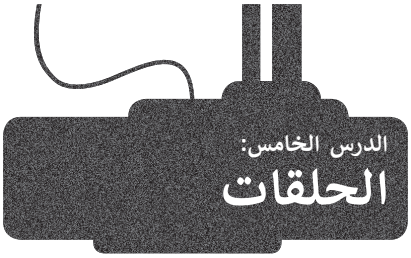
المخطط الانسيابي للمقطع البرمجي

المقطع البرمجي

وزارة التعليم
Ministry of Education
2026 - 1448

رابط الدرس الرقمي

www.ien.edu.sa



ستحتاج في بعض الأحيان إلى تكرار مجموعة من الأوامر عدة مرات في النص البرمجي، وسيتطلب هذا التكرار الكثير من الوقت والجهد. لذلك، توفر جميع لغات البرمجة تقريبًا بنية تحكم تسمى **حلقة** (loop). ستسمح لك هذه الدالة بتنفيذ صف واحد أو مجموعة من المقاطع البرمجية عدة مرات. ويمكنك تحديد عدد التكرارات برقم معين، أو أن تعتمد على شرط معين.

يدعم بايثون نوعين من الحلقات: حلقة **for** و حلقة **while**.

الحلقات في بايثون

حلقة for

```
for loop_variable in range():
    statements
```

حلقة while

```
while condition:
      statements
```

حلقة for

تُستخدم حلقة **for** لتكرار مجموعة من الأوامر لعددٍ محددٍ من المرات، ويكون عدد التكرارات محدَّدًا في قيم دالة **النطاق** (range()).

```
for loop_variable in range():
    statements
```

يجب أن تُسبق البيانات المتكررة بمسافة بادئة.

يمكنك استخدام دالة **النطاق** لتحديد عدد مرات تنفيذ حلقة **for**.

المسافة البادئة في الحلقات

كما تعلمت سابقًا، فإن استخدام **المسافة البادئة** (Indentation) لعبارات IF الشرطية، هو أمر مهم جدًا في بايثون. يتعرّف المقطع البرمجي على العبارات المضمنة في الحلقة والتي يتم تنفيذها في كل تكرار لها، من خلال المسافة البادئة، لذلك إذا لم تترك مسافة بادئة في مقطعك البرمجي، فستتلقى رسالة خطأ.

دالة النطاق

تُستخدم دالة **النطاق** (( )range) مع الحلقة لتحديد عدد التكرارات، و يسمى المتغير الذي يحسب التكرار **العدّاد** (counter).

في دالة **النطاق** يبدأ العداد بالعد من 0، يزيد بمقدار 1 وينتهي العد قبل الوصول إلى الرقم المحدد. مثال على ذلك:

```
# i يطبع قيمة
for i in range(5):
    print(i)
```

```
0
1
2
3
4
```

يمكن تحديد مقدار البداية بإضافة قيمة. على سبيل المثال، النطاق (2,5)، يعني أن العداد يبدأ من 2 وينتهي عند 4 (القيمة 5 غير مدرجة).

```
# i يطبع قيمة
for i in range(2,5):
    print(i)
```

```
2
3
4
```

القيمة التلقائية للزيادة في دالة النطاق هي 1، ولكن يمكن تحديد مقدار الزيادة بإضافة قيمة ثالثة. على سبيل المثال، النطاق (1,5,2)، يعني أن العداد يبدأ في العد من 1، وينتهي عند 4 ويزداد بمقدار 2.

```
# i يطبع قيمة
for i in range(1,5,2):
    print(i)
```

```
1
3
```

```
ut the valu
range(1,5,2):
i)
```

القيمة الثالثة في دالة النطاق يسمى **الخطوة** (the step).

يمكن أن تكون الخطوة عددًا سالبًا، ويصبح العدُّ بشكلٍ عكسي.

```
# i يطبع قيمة
for i in range(10,5,-2):
    print(i)
```

```
10
8
6
```


وزارة التعليم
Ministry of Education
2026 - 1448



جرب المقطع البرمجي التالي واكتب القيم التي تظهر على الشاشة.

```
for i in range(4,0,-1):
    print (i)
```

```
for i in range(0,10,2):
    print (i)
```

مثال: درجات تقييم الطلبة

لقد عملت سابقًا على مثال لحساب درجات تقييم الطلبة، حيث يتحقق المقطع البرمجي من اجتياز الطالب للامتحانات. ستتعلم كيف يمكنك تطبيق حلقة **for**. لقد تحققت في المثال السابق من درجات طالب واحد فقط، لتفترض أن عليك التحقق من درجات صف كامل مكون من 15 طالبًا.

لا تنسَ ترك مسافة.

```
for st in range (0,15):
    print("الرجاء إدخال اسم الطالب: ")
    name=input()
    print("الرجاء إدخال درجة الطالب: ")
    g=int(input())
    if g>50:
        print(name,"اجتاز الامتحانات.")
        if g<=70:
            print(name,"حصل على C.")
        elif g<=90:
            print(name,"حصل على B.")
        else:
            print(name,"حصل على A.")
    else:
        print(name,"لم يجتز الامتحانات.")
```

لقراءة اسم كل طالب.

تُستخدم حلقة **for** عند معرفة عدد التكرارات المراد قبل بداية التكرار. ماذا يحدث عندما يكون الرقم غير معروف والتكرار يعتمد على شرط معين؟ في مثل هذه الحالات، تُقدم بايثون حلقة **while** الشرطية.

حلقة while الشرطية

تُستخدم حلقة **while** عندما لا يكون عدد التكرارات معروفًا سابقًا. طالما أن الحالة صحيحة، فإن الحلقة تتكرر وتُفحص بعد كل تكرار للتأكد من صحتها. عندما تُصبح الحالة خطأ، يتوقف التكرار وينتقل المقطع البرمجي إلى الصف التالي بعد الحلقة. أما إذا كانت حالة الشرط خطأً في البداية، فلن يتم تنفيذ الحلقة على الإطلاق.

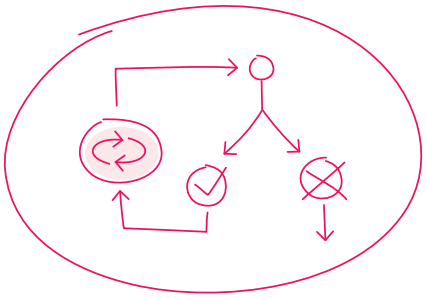
```
while condition:
    statements
```

يجب أن تُسبق الأوامر المتكررة بمسافة بادئة.

انظر إلى مثال حلقة **while** الشرطية، يُدخل المستخدم قيمة للمتغير x، وتنتهي الحلقة عندما يُدخل المستخدم 0 كقيمة للمتغير x.

```
x=int(input("أدخل قيمة x: "))
while x!=0:
    print(x)
    x=int(input("أدخل قيمة x: "))
print("نهاية الحلقة.")
```

```
أدخل قيمة x: 5
5
أدخل قيمة x: 6
6
أدخل قيمة x: 10
10
أدخل قيمة x: 0
نهاية الحلقة.
```



وزارة التعليم
Ministry of Education
2026 - 1448

يُمكنك استخدام حلقة **while** الشرطية للتحقق من مدخلات المستخدم في متغير معين.

مثال: درجات تقييم الطلبة

في هذا المثال، سيطلب المقطع البرمجي من المستخدم إدخال درجة الطالب، ويجب أن تكون هذه الدرجة أكبر من أو تساوي 0، وأقل من أو تساوي 20. في حال أدخل المستخدم قيمة خارج هذا النطاق، فسيعرض المقطع البرمجي رسالة خطأ ويطلب من المستخدم إدخال درجة صالحة.

```
# يجب أن تكون درجات الطلبة أكبر من أو تساوي 0
# وأقل من أو تساوي 20
grade=int(input("أدخل درجة الطالب: "))
while grade<0 or grade>20:
    print("درجة غير صالحة، أدخل درجة من ٠-٢٠.")
    grade=int (input("أدخل درجة صالحة: "))
print ("درجتك هي: ", grade)
```

```
أدخل درجة الطالب: 67
درجة غير صالحة، أدخل درجة من 0-20.
أدخل درجة صالحة: 18
درجتك هي: 18
```



جرب المقطع البرمجي التالي واكتب ما يظهر على الشاشة.

```
i=1
while i<6:
  i=i+1
  if i == 3:
    print("مرحبًا")
  print(i)
```

حلقة لا نهائية

إذا لم يُصبح شرط حلقة **while** خطأ، فسينتهي بك الأمر **بحلقة لا نهائية** (Infinite loop)، وهي حلقة لا تنتهي أبدًا.

عندما تستخدم حلقة **while** الشرطية، يجب أن تتضمن أمرًا، أو مجموعة من الأوامر التي ستغير الحالة من **صواب** (True) إلى **خطأ** (False).



جرب المقطع البرمجي التالي. ماذا تلاحظ؟

```
i=1
while i<6:
    print(i)
```

في المثال السابق، قيمة المتغير i لا تتغير، لذلك سيتم تكرار المقطع البرمجي إلى الأبد.

لإيقاف تكرار الحلقة، اضغط على Ctrl + C في نافذة Python shell.

وزارة التعليم
Ministry of Education
2026 - 1448

عبارة الإيقاف

في بعض الأحيان تريد إنهاء حلقة قبل أن تصبح حالتها خطأ. في مثل هذه الحالات، ستستخدم **عبارة الإيقاف** (Break statement). تُنهي عبارة الإيقاف الحلقة التي تحتوي عليها، وينتقل المقطع البرمجي إلى الصف المتواجد بعد الحلقة. يمكن استخدام عبارة الإيقاف في حلقة **for** أيضًا.

عادة ما يكون هناك العديد من الطرق المختلفة لأداء نفس المهمة.

تُفضل إحدى الطرق على الأخرى بناءً على عدة عوامل، أهمها سرعة تشغيل المقطع البرمجي ومساحة التخزين المطلوبة.

المبرمج يُحدد أفضل طريقة.

```
while True:
    word=input("اكتب كلمة: ")
    if word=="إيقاف":
        print("لقد استخدمت عبارة الإيقاف.")
        break
    print("اكتب كلمة مختلفة")
```

```
اكتب كلمة: سيارة
اكتب كلمة مختلفة
اكتب كلمة: طائرة
اكتب كلمة مختلفة
اكتب كلمة: دراجة
اكتب كلمة مختلفة
اكتب كلمة: دراجة هوائية
اكتب كلمة مختلفة
اكتب كلمة: إيقاف
لقد استخدمت عبارة الإيقاف.
```

إن السلاسل النصية في بايثون حساسة لحالة الأحرف، لذلك عند التحقق من قيمة متغير، عليك أيضًا التحقق مما إذا كانت هناك مسافات زائدة؛ لأن المسافة تُعتبر حرفًا في بايثون أيضًا.

لنطبق معًا

تدريب 1

كم مرة سينفذ أمر الطباعة (print())؟ اختر الإجابة الصحيحة:

اختر الإجابة الصحيحة		
لن ينفذ؛ لأن تنسيق الأوامر غير صحيح.	○	`for i in range (0,5,3):` `  print(i)`
ينفذ مرتان.	○	
ينفذ 3 مرات.	○	
ينفذ 5 مرات.	○	`for i in range (10,1,-2):` `  print(i)`
عدد المرات غير معروف.	○	
ينفذ 4 مرات.	○	

ينفذ 5 مرات.	○	`i=5`
		`while i>1:`
		`  print(i)`
		`  i=i-1`

عدد المرات غير معروف.	○	
ينفذ 4 مرات.	○	

وزارة التعليم
Ministry of Education
2026 - 1448

تدريب 2

اكتب مقطعًا برمجيًا يعرض الأرقام من 10- إلى 1- باستخدام حلقة for.

تدريب 3

اكتب مقطعًا برمجيًا يعرض الأرقام 100، 95، 90،...، 0 على الشاشة.

تدريب 4

اكتب مقطعًا برمجيًا باستخدام حلقة while لعرض أول 13 عدد من مضاعفات العدد 7.

تدريب 5

اكتب مقطعًا برمجيًا لحساب مجموع جميع الأرقام من 1 إلى رقم معطى.

- اطلب من المستخدم إدخال رقم.
- احسب مجموع جميع الأرقام من 1 إلى الرقم المعطى.
- على سبيل المثال، إذا أدخل المستخدم 8، فيجب أن يكون الناتج 36 (1 + 2 + 3 + 4 + 5 + 6 + 7 + 8).

رابط الدرس الرقمي

www.ien.edu.sa



لقد تعرّفت على الحلقات في الدرس السابق وتعلّمت كيفية استخدام أنواعها المختلفة. في هذا الدرس ستتعلم كيفية وضع حلقة داخل حلقة أخرى، ويسمى هذا الإجراء **بالتداخل** (Nesting) كما تسمى هذه الحلقات **بالحلقات المتداخلة** (Nested loops)، حيث يمكنك إدخال أي نوع من الحلقات في أي نوع آخر، وعلى سبيل المثال: يمكنك إدخال حلقة for في حلقة while أو العكس. وتعدّ حلقات for هي الحلقات المتداخلة الأكثر شيوعًا.

الحلقة الخارجية

الحلقة المتداخلة تُعني جملة حلقة داخل جملة حلقة أخرى.

الحلقة الداخلية

```
for i in range(3):
    for j in range(2):
        print("i= ",i,"j= ",j)
```

```
i= 0 j= 0
i= 0 j= 1
i= 1 j= 0
i= 1 j= 1
i= 2 j= 0
i= 2 j= 1
```

خطوات التنفيذ:

1. i يحصل على القيمة 0، ستتكرر الحلقة الداخلية مرتين: for j = 0 ، j = 1.
2. الآن ارفع قيمة i، ثم for i = 1، ستتكرر الحلقة الداخلية مرتين: for j = 0 ، j = 1.
3. الآن ارفع قيمة i، ثم for i = 2، ستتكرر الحلقة الداخلية مرتين: for j = 0 ، j = 1.

في النهاية، تكررت الحلقة الخارجية 3 مرات والحلقة الداخلية 6 مرات.

القيم	
i	j
0	0
	1
1	0
	1
2	0
	1

1 2 3

وزارة التعليم
Ministry of Education
2026 - 1448

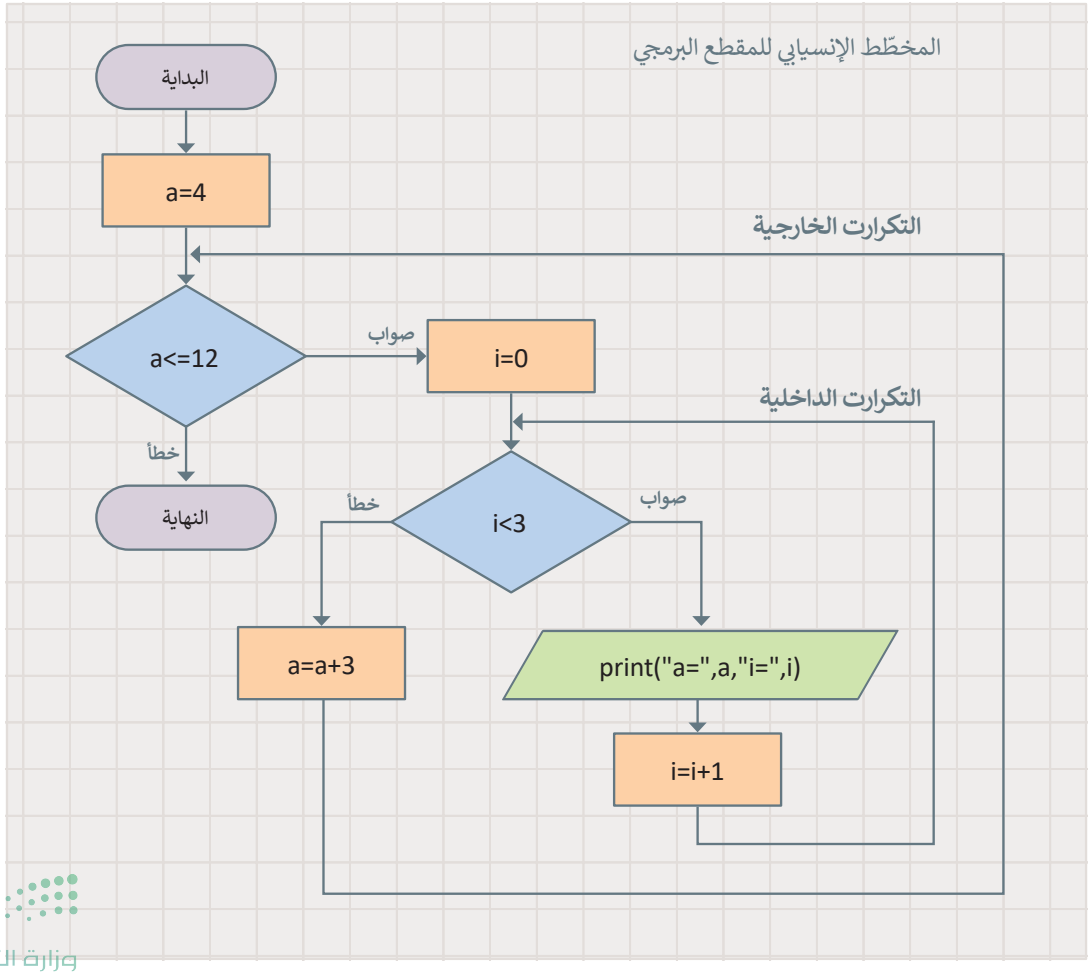
مثال آخر للحلقات المتداخلة.

```
a=4
while a<=12:
    for i in range(3):
        print("a= " ,a,"i= ",i)
    a=a+3
```

```
a= 4 i= 0
a= 4 i= 1
a= 4 i= 2
a= 7 i= 0
a= 7 i= 1
a= 7 i= 2
a= 10 i= 0
a= 10 i= 1
a= 10 i= 2
```

وضع المسافة البادئة في الحلقات المتداخلة مهم جدًا، حيث تُحدد المسافة البادئة الأوامر التي يتم تضمينها في كل حلقة.

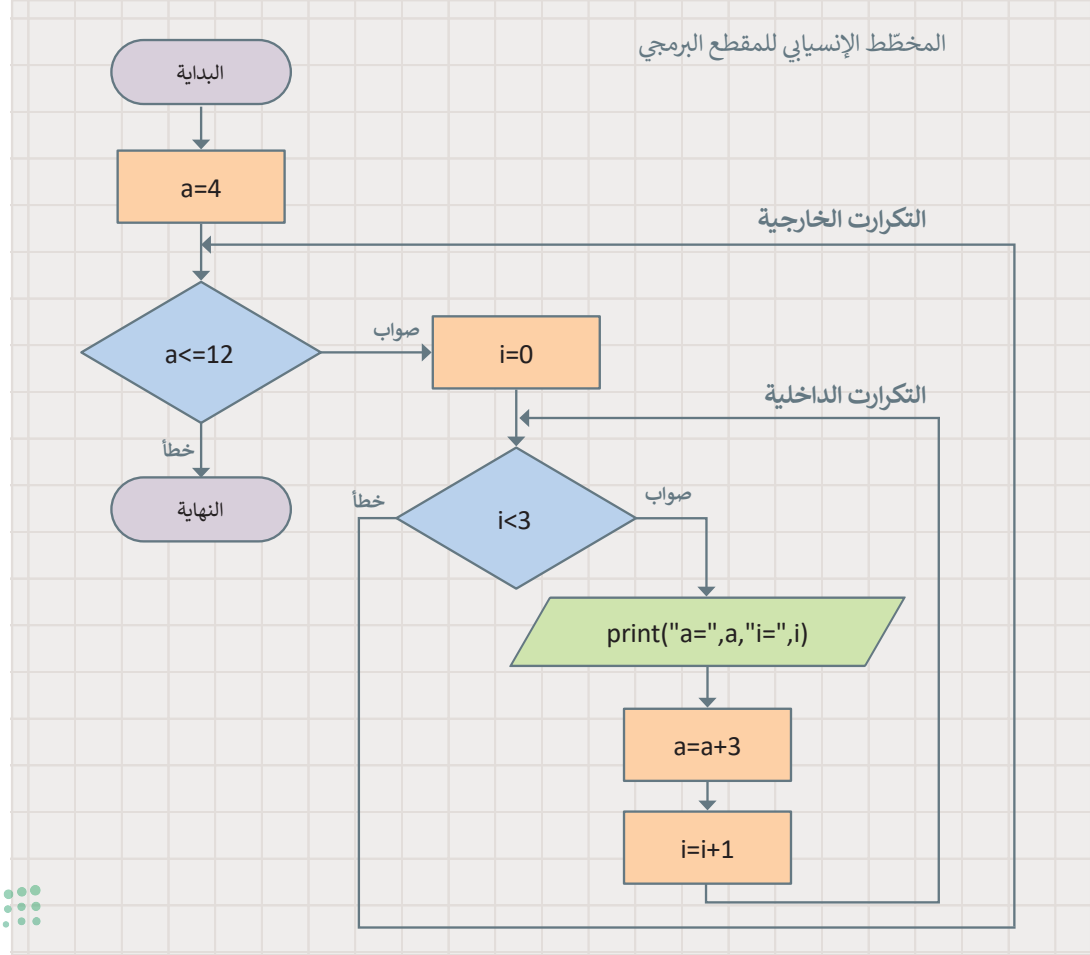
المخطّط الإنسيابي للمقطع البرمجي



وضع المسافة البادئة مهم للغاية ويغير المقطع البرمجي بأكمله في بايثون. يمكنك أن ترى أنه إذا غيرت المسافة البادئة في المثال السابق، فإن النتيجة ستكون مختلفة.

```
a=4
while a<=12:
    for i in range(3):
        print("a= ",a,"i= ",i)
        a=a+3
```

```
a=  4 i=  0
a=  7 i=  1
a=  10 i=  2
```



وزارة التعليم
Ministry of Education
2026 - 1448

القواعد التي تنطبق على الحلقات المتداخلة:

> يجب أن تكون كل حلقة داخلية مدمجة بالكامل داخل الحلقة الخارجية، ولا يمكن أن تتشابك الحلقات.

> لا يمكن استخدام نفس المتغير كعدّاد لحلقتين متداخلتين أو أكثر.

> الحلقة الداخلية يجب أن تكتمل أولًا.

> الحلقة الداخلية تنفذ جميع تكراراتها لكل تكرار من تكرارات الحلقة الخارجية.

مثال توضيحي

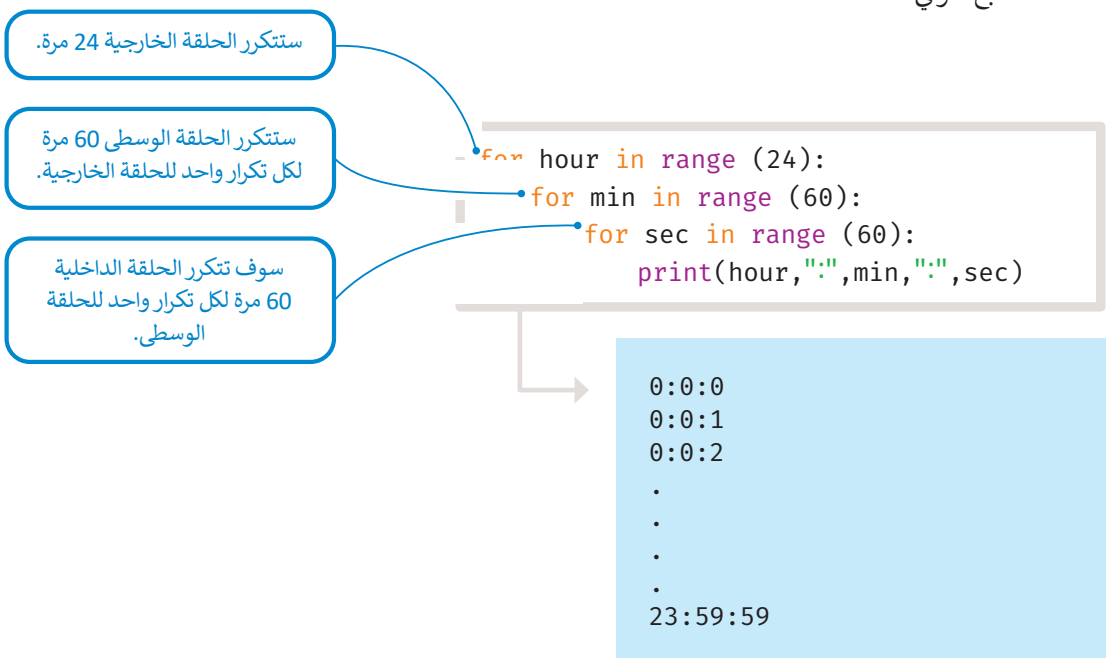
الساعة الرقمية هي إحدى أمثلة الحلقات المتداخلة في العالم الحقيقي.

في الساعة الرقمية، تحتاج إلى 3 حلقات:

> الحلقة الأولى ستتبع الساعات.

> الحلقة الثانية ستتبع الدقائق.

> الحلقة الثالثة ستتبع الثواني.



مثال: حساب درجات تقييم الطلبة

المعلم يريد حساب درجات تقييم طلبته.

يضم فصله 30 طالبًا، وخضع كل طالب لاختبارين وامتحان نهائي واحد.
الدرجة النهائية لكل طالب هي متوسط تلك الدرجات الثلاث.

```
# الحلقة الخارجية للطلبة ال 30
for student in range(30):
    name=input("اكتب اسم الطالب: ")
    # ابدأ بكتابة الدرجات الإجمالية لكل طالب
    sumGrades=0
    # حلقة داخلية للدرجات الثلاث
    for gr in range(1,4):
        print("اكتب الدرجة",gr," للطالب: ")
        grade=int(input())
        #مجموع الدرجات الثلاث للطالب
        sumGrades=sumGrades+grade
    # احسب الدرجة النهائية
    finalGrade=sumGrades/3
    print("الدرجة النهائية ل",name,"هي",finalGrade)
```

```
اكتب اسم الطالب: محمد
اكتب الدرجة 1 للطالب:
75
اكتب الدرجة 2 للطالب:
80
اكتب الدرجة 3 للطالب:
90
الدرجة النهائية ل محمد هي 81.66666666666667
اكتب اسم الطالب:
```


وزارة التعليم
Ministry of Education
2026 - 1448

أنماط الطباعة

يمكنك استخدام بايثون لعرض الأنماط على الشاشة. لطباعة أي نمط، هناك هيكل عام عليك اتباعه، حيث تحتاج إلى تحديد عدد الصفوف والأعمدة في النمط. تُخبرك الحلقة الخارجية بعدد الصفوف المستخدمة، تخبرك الحلقة الداخلية بعدد الأعمدة المستخدمة لطباعة النمط.

بعض الأمثلة على الأنماط الرقمية:

```
for num in range (1,6):
    for j in range (num):
        print(num,end=" ")
    # الانتقال لصف جديد
    print(" ")
```

استخدام end بهذه الطريقة يختم النص المطبوع بمسافة فارغة بدلًا من الانتقال إلى صف جديد.

```
1
2 2
3 3 3
4 4 4 4
5 5 5 5 5
```

في المثال أعلاه، الحلقات الخارجية والداخلية من نوع حلقة **for**، حيث تعمل الحلقة الخارجية في النطاق (1,6) و تعمل الحلقة الداخلية داخل النطاق المحدد للحلقة الخارجية. سيتم عرض الرقم الأول مرة واحدة في الصف الأول، والرقم الثاني مرتين في الصف الثاني، والرقم الثالث ثلاث مرات في الصف الثالث وهكذا.

```
for num in range (1,6):
    for j in range (1,num+1):
        print(j,end=" ")
    # الانتقال لصف جديد
    print(" ")
```

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```



جرب المقطع البرمجي التالي، واكتب النتيجة التي تظهر على الشاشة.

```
for num in range (8,-1,-1):
    for j in range (0, num):
        print(num,end=" ")
    # الانتقال لصف جديد
    print()
```

وزارة التعليم
Ministry of Education
2026 - 1448

في المثال التالي، سترى أنه يمكنك طباعة أنماط هرمية من النجوم. ستطبع نجمة واحدة في الصف الأول، ونجمتين في الصف الثاني وتستمر بالطباعة حتى الصف الخامس.

تريد تنفيذ الحلقة المتداخلة عدد مرات i، لذا فأنت بحاجة إلى i + 1 في دالة النطاق.

```
    for j in range(0, i+1):
        print("*",end=" ")
    print()
```

```
*
* *
* * *
* * * *
* * * * *
```

يمكنك أيضًا رسم مثلث باستخدام النجوم، حيث ستستخدم 4 حلقات **for**، حلقتين داخليتين مخصصتين لتكرار العمود، وحلقتين خارجيتين لتكرار الصف.

```
for i in range(6):
    for j in range(i):
        print ("*", end=" ")
    print()

for i in range(6,0,-1):
    for j in range(i):
        print("*", end=" ")
    print()
```

```
*
* *
* * *
* * * *
* * * * *
* * * * * *
* * * * *
* * * *
* * *
* *
*
```



جرب المقطع البرمجي التالي، واكتب النتيجة التي تظهر على الشاشة.

```
for i in range(6, 0, -1):
    for j in range(0, i - 1):
        print("*", end=" ")
    print()
```


وزارة التعليم
Ministry of Education
2026 - 1448

لنطبق معًا

تدريب 1

شغّل المقطع البرمجي التالي وأكمل الجدول.

```
x=2
c=5
while c>0:
    for i in range (7,12,2):
        x=x+3
        print(i,x)
    c=c-3
    print(x,c)
```

جدول القيم

الشاشة	i	c	x
		5	2
7 5	7		5
		2	
7 14	7		
-1 20			

تدريب 2

شغّل المقطع البرمجي التالي وتحقق من عدد الصف المطبوعة.

```
for i in range(4):
    for j in range(8):
        print(i, j)
```

- A. 4
- B. 5
- C. 12
- D. 24
- E. 32

وزارة التعليم
Ministry of Education
2026 - 1448

تدريب 3

اكتب مقطعًا برمجيًا لعرض جدول الضرب لجميع الأرقام من 1 إلى 10.

تدريب 4

ما وظيفة المقطع البرمجي التالي؟ وما مخرجاته؟

```
for i in range(3):
    for j in range(3):
        if j == i:
            break
        print(i, j)
```

الناتج:



.....

.....

.....

.....

.....

.....

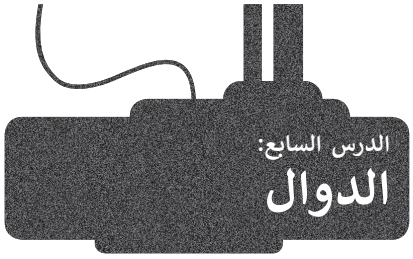
وزارة التعليم
Ministry of Education
2026 - 1448

تدريب 5

اكتب المقطع البرمجي الذي يطبع النمط التالي:

```
5 4 3 2 1
4 3 2 1
3 2 1
2 1
1
```

المقطع البرمجي:



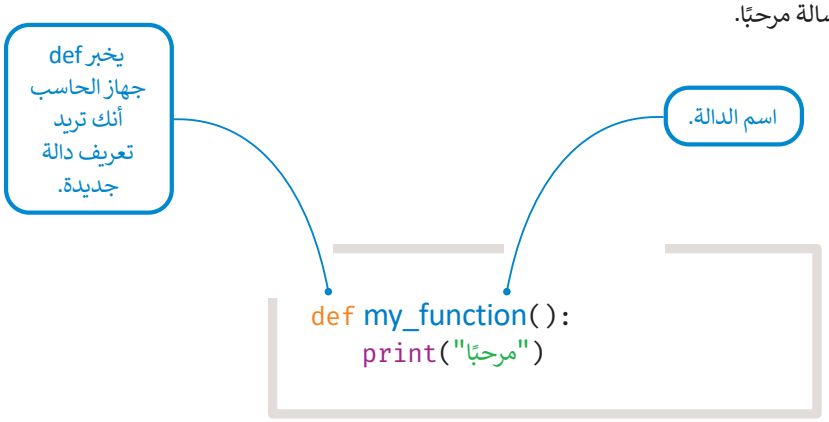
رابط الدرس الرقمي
www.ien.edu.sa

الدالة (Function) مجموعة من الأوامر التي يتم تجميعها في مكان واحد مع إعطائها اسمًا (تعريفًا)، ويتم تنفيذها من خلال استدعائها عند الحاجة لها. ويحتوي البايثون على مجموعة من الدوال الجاهزة والتي سبق لك استخدامها، مثل: دالة **الطباعة** (print())، ودالة **الطول** (len())، ودالة **النطاق** (range())، ودالة **الإدخال** (input()). في هذا الدرس ستتعلم كيفية إنشاء دوال جديدة لتساعدك في بعض المهام والمشكلات المتكررة.

إنشاء الدوال الخاصة بك

في بايثون، يُعرّف الجزء الأول من الدالة باستخدام الكلمة الأساسية **def** ويحتوي هذا الجزء على **تعريف الدالة** (function definition)، ثم يأتي الجزء الثاني وهو **اسم** الدالة (name). في نهاية **رأس الدالة** (function header)، هناك **نقطتان** (colon) للإشارة إلى أن ما يلي عبارة عن مجموعة من المقاطع البرمجية، ويجب وضع مسافة بادئة لها. بعد ذلك، يتبع **محتوى الدالة** (function body)، والذي في هذا المثال، يطبع الرسالة فقط.

الآن ستحدد دالة تطبع رسالة مرحبًا.



استدعاء دالة

لاستدعاء دالة، استخدم اسم الدالة متبوعًا بأقواس.

ومن الأمثلة على الدالة ما يلي:

```
def my_function():
    print("مرحبًا")

# function call
my_function()
```

مرحبًا

الميزة الرئيسة للدوال هي أنه يمكنك استدعاؤها مرارًا وتكرارًا من المقطع البرمجي الرئيس.

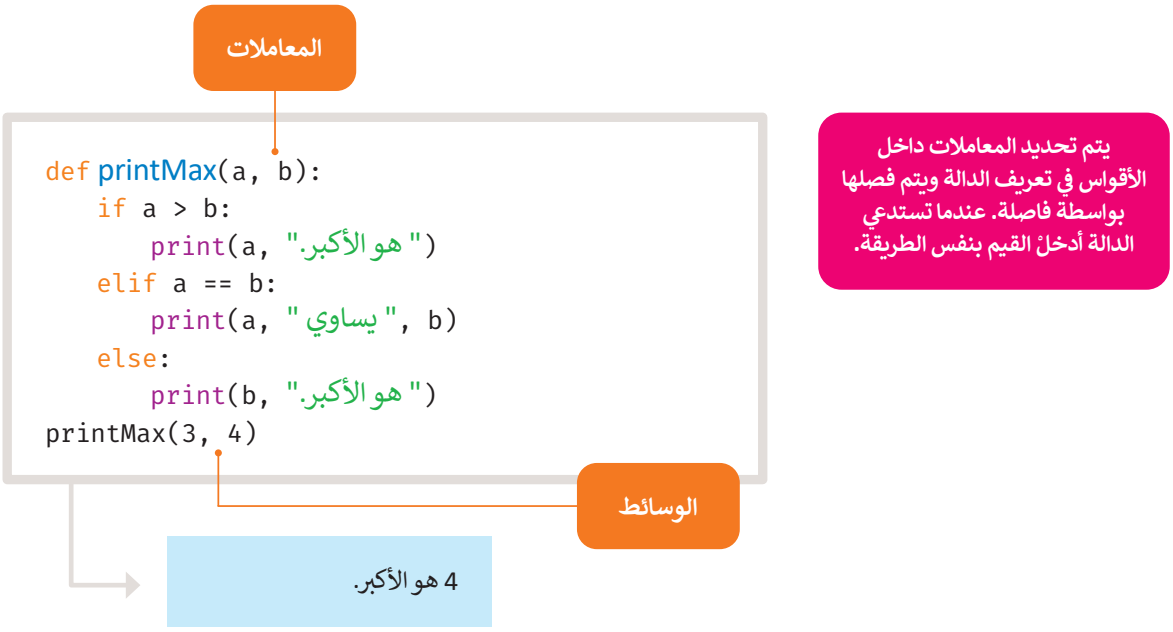
وزارة التعليم
Ministry of Education
2026 - 1448

المُعامِلات والوسائط

المتغيرات التي يمكنك الإعلان عنها في الدالة تُسمى **المُعامِلات** (Parameters)، وتُستخدم هذه المتغيرات داخل الدالة.

ونظرًا لأنه لا يمكن الوصول للمعاملات إلا من خلال الدالة، فإنه يُشار إليها باسم **المتغيرات المحلية** (Local Variables)، كما يُشار إلى المتغيرات التي تم تمريرها إلى الدالة لتنفيذها على أنها **وسائط** (Arguments). وتتلقى المتغيرات المحلية للدالة قيم الوسائط كمُدخلات ثم تتابع تنفيذ الدالة.

ألقِ نظرة على المثال الآتي، حيث تحدد دالة تسمى **printMax** مع معاملين **a** و **b** وتجد أكبر عدد باستخدام جملة **if else** الشرطية، ثم تطبع هذا العدد باستخدام دالة **الطباعة** (print()).





جرب المقطع البرمجي التالي، واكتب النتيجة التي تظهر على الشاشة.

```
def profileMessage(age, name):
    print("اسمي",name,"عمري", age)

profileMessage(15,"ريم")
```

وزارة التعليم
Ministry of Education
2026 - 1448

عبارة الإرجاع

في بايثون، تُستخدم **عبارة الإرجاع** (Return statement) لإنهاء تنفيذ استدعاء الدالة و إرجاع قيمة التعبير. يؤدي ظهور عبارة الإرجاع في أي نقطة من محتوى الدالة إلى إنهاء تنفيذ أوامر الدالة وإعادة المقطع البرمجي إلى النقطة التي تلي أمر استدعائه مباشرة.

لا يمكن استخدام عبارة الإرجاع خارج الدالة.

```
def squareRoot(a):
    return a*a
# نطع البرمجي الرئيس
finalNumber = s
print(finalNumb
```

```
25
```

تعطي عبارة الإرجاع **return** قيمة ما أو ردًا على مستخدم الدالة، بينما ينتج عن أمر الطباعة **print** نصًا مطبوعًا.

في المثال أدناه، تُرجع دالة **الحد الأقصى** (maximum) أكبر القيم التي تم إدخالها، وللقيام بذلك تُستخدم جملة **If else** الشرطية للعثور على القيمة الأعلى ثم إرجاع تلك القيمة.

في حال عدم إضافة عبارة إرجاع إلى دالتك، فإن الدالة لا تعيد أي شيء.

```
def maximum(x, y):
    if x > y:
        return x
    elif x == y:
        return "الأرقام متساوية."
    else:
        return y

print(maximum(2, 3))
```

```
3
```


وزارة التعليم
Ministry of Education
2026 - 1448



جرب المقطع البرمجي التالي، واكتب النتيجة التي تظهر على الشاشة.

```
def f(x):

  return 5 * x

print(f(2))
print(f(5))
print(f(6))
```

الوسائط الافتراضية

في بعض الدوال، قد ترغب في جعل بعض المعاملات اختيارية واستخدام القيم الافتراضية إذا كان المستخدم لا يريد إعطاء قيم لهذه المعاملات. يمكن تحقيق ذلك بمساعدة قيم الإعداد الافتراضي.

إذا لم يتم تمرير قيمة أخرى أثناء تسمية الدالة، فستأخذ وسائط الدالة قيم الإعداد الافتراضية.

يمكنك تعيين وسائط افتراضية (Default arguments) للمعاملات عن طريق وضع عامل الإحالة (=) متبوعًا بالقيمة الافتراضية بعد تسمية المعاملات في تعريف الدالة.

القيمة افتراضية

```
def helloFunction(message="مرحبًا"):
    print(message

helloFunction()
helloFunction(message="مرحبًا من بايثون!")
```

إذا تم استدعاء الدالة بدون وسائط، تُستخدم القيمة الافتراضية.

إذا كنت ترغب في وضع قيمة مختلفة، يمكنك القيام بذلك كما هو موضح.

```
مرحبًا
مرحبًا من بايثون!
```

المتغيرات المحلية والعامة

عند تحديد المتغيرات داخل تعريف الدالة، فإنها لا تؤثر ولا تتأثر بالمتغيرات الأخرى التي تحمل نفس الاسم والتي يتم استخدامها خارج تلك الدالة. يسمى جزء المقطع البرمجي الذي يُمكن الوصول إلى المتغير واستخدامه بنطاق المتغير، **المتغيرات المحلية** (Local Variables) نطاقها داخل الدالة حيث تم التصريح عنها، وتبدأ من تعريف اسم الدالة، وتنتهي عند توقف الدالة عن التنفيذ.

في المثال التالي دالة استخدمت لأول مرة معامل يسمى x، يستخدم بايثون قيمة المعامل المعلن داخل تلك الدالة.

ثم ستسند القيمة 2 إلى x، والاسم x هو متغير محلي لدالتك. لذلك، عندما تغيرت قيمة x في الدالة، فإن x المحددة في الجزء الرئيس لن تتأثر.

في الاستدعاء الأخير لدالة الطباعة، ستُطبع قيمة x في القسم الرئيس وتؤكد على أنها لم تتأثر.

```
x = 50

def func(x):
    print("x= ", x)
    x = 2
    print("لقد غيرت x إلى", x)

func(x)
print("لا تزال x=", x)
```

```
x= 50
لقد غيرت x إلى 2
لا تزال x= 50
```

إذا كنت ترغب في تعيين قيمة لاسم محدد في المستوى الأعلى من جزء المقطع البرمجي (أي ليس ضمن نطاق معين مثل الدوال أو الفئات)، فأنت بحاجة إلى إخبار بايثون أن الاسم ليس **محليًا** (local) ولكنه **عام** (global). يتم ذلك باستخدام **المتغيرات العامة** (Global Variables).

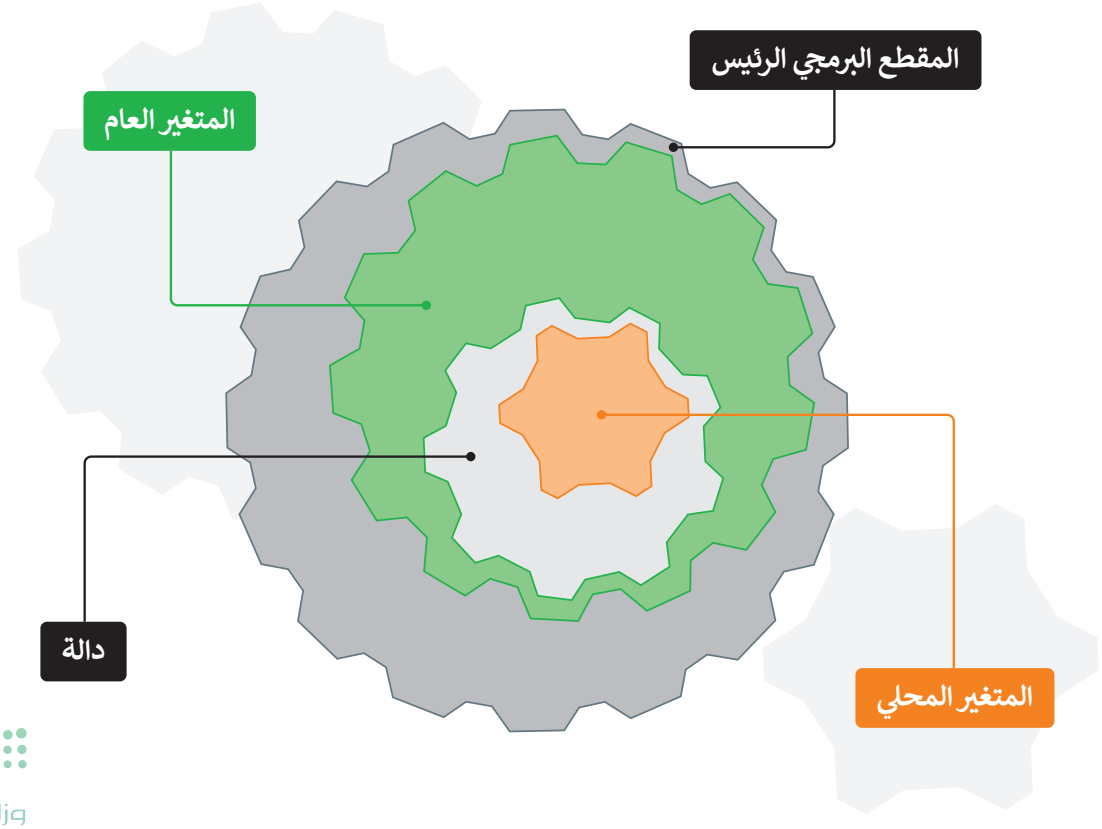
وزارة التعليم
Ministry of Education
2026 - 1448

في المثال التالي يستخدم فيه الأمر **العام** (global) للإشارة إلى أن x هو متغير عام، حيث يؤدي إسناد قيمة إلى x داخل الدالة لتغيير قيمة x في الجزء الرئيس.

```
x = 50
def func():
    global x
    print("قيمة x هي", x)
    x = 2
    print("لقد غيرت x العامة إلى", x)

func()
print("قيمة x هي", x)
```

```
قيمة x هي 50
لقد غيرت x العامة إلى 2
قيمة x هي 2
```



لنطبق معًا

تدريب 1

اكتب ناتج المقطع البرمجي التالي:

```
total = 0;
# تعريف الدالة هنا
def sum( arg1, arg2 ):
    total = arg1 + arg2;
    print("المجموع داخل الدالة: ", total)
    return total;

# المقطع البرمجي الرئيس
sum( 10, 20 );
print("المجموع خارج الدالة: ", total)
```

الناتج

وزارة التعليم
Ministry of Education
2026 - 1448

تدريب 2

المقطع البرمجي التالي غير مكتمل. عليك أن تكتب دالة لإيجاد متوسط الأعداد الثلاثة التي أدخلتها في المقطع البرمجي الرئيس.

```
def




المقطع البرمجي الرئيس #

num1 = float(input("أدخل الرقم الأول: "))
num2 = float(input("أدخل الرقم الثاني: "))
num3 = float(input("أدخل الرقم الثالث: "))
result = find_average(num1, num2, num3)
print("متوسط الأرقام هو: ", result)
```

تدريب 3

اشرح الفرق بين المتغيرات المحلية والعامة.

.....

.....

.....

.....

.....

.....

تدريب 4

اكتب دالة في بايثون تقبل ثلاثة أرقام وتعيد الرقم الأكبر من بين هذه الأرقام.

.....

.....

.....

.....

.....

.....

تدريب 5

اكتب مقطعًا برمجيًا ودالة من أجل طباعة رسالة، على سبيل المثال، "مرحبًا بك في المدرسة يا خالد". اسم المستخدم هو معامل البرنامج.

> أنشئ المقطع البرمجي الرئيس الذي يحدد قيمة الوسائط والدالة التي تطبع الرسالة.

> أنشئ مقطعًا برمجيًا آخر يُدخل فيه المستخدم قيمة المعاملات في الدالة، ويطبع المقطع البرمجي الرئيس الرسالة.

تدريب 6

اكتب مقطعًا برمجيًا يسأل الطالب عن درجاته في أربع مواد، ثم أنشئ دالة تأخذ هذه الدرجات كوسائط ويحسب ما إذا كان الطالب قد اجتاز الفصل أم لا.

لاجتياز الفصل، يجب أن يكون لدى الطالب متوسط أعلى من 12.

يجب أن تكون الدرجات أعدادًا صحيحة أكبر من 0 وأقل من 21.

وزارة التعليم
Ministry of Education
2026 - 1448

رابط الدرس الرقمي

www.ien.edu.sa

الدرس الثامن:

جداول بيانات إكسل في بايثون

لقد تعلمت سابقًا كيفية استخدام إكسل لإجراء معالجة البيانات البسيطة، مثل إنشاء جداول البيانات، وتطبيق الدوال لتحديد البيانات وفرزها وتنسيقها وإنشاء بعض المخططات بناءً على بياناتك. ستتعلم في هذا الدرس كيفية استخدام بايثون إلى جانب إكسل.

العمل مع إكسل و بايثون

نظرًا لأنك تعلم بالفعل كيفية استخدام إكسل، فقد تتساءل عن سبب استخدام بايثون للعمل معه. السبب هو أنه باستخدام بايثون يمكنك أتمتة بعض المهام المتكررة، وإجراء بعض العمليات الحسابية المعقدة بشكل أسرع من العمل مع إكسل فقط.

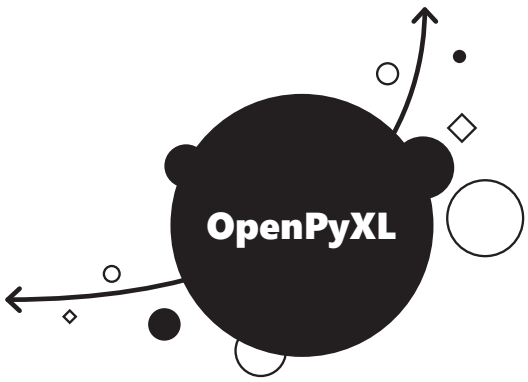
لاستخدام بايثون مع إكسل، تحتاج إلى مكتبة تسمى **أوبين بيكسل** (openpyxl). المكتبة في لغات البرمجة هي عبارة عن حزمة مقاطع برمجية مكتوبة سابقًا، وتؤدي بعض المهام بحيث لا يحتاج المبرمجون إلى كتابة المقطع البرمجي من البداية في كل مرة.

مكتبة أوبين بيكسل

يوجد في بايثون العديد من المكتبات المثبتة سابقًا، ولكن هناك العديد من المكتبات المتاحة التي يمكنك تنزيلها وتثبيتها، مكتبة **أوبين بيكسل** (openpyxl) هي إحدى تلك المكتبات. يمكنك باستخدامها إجراء عمليات مثل القراءة والكتابة والحسابات الرياضية، وإنشاء الرسوم، والمخططات البيانية.







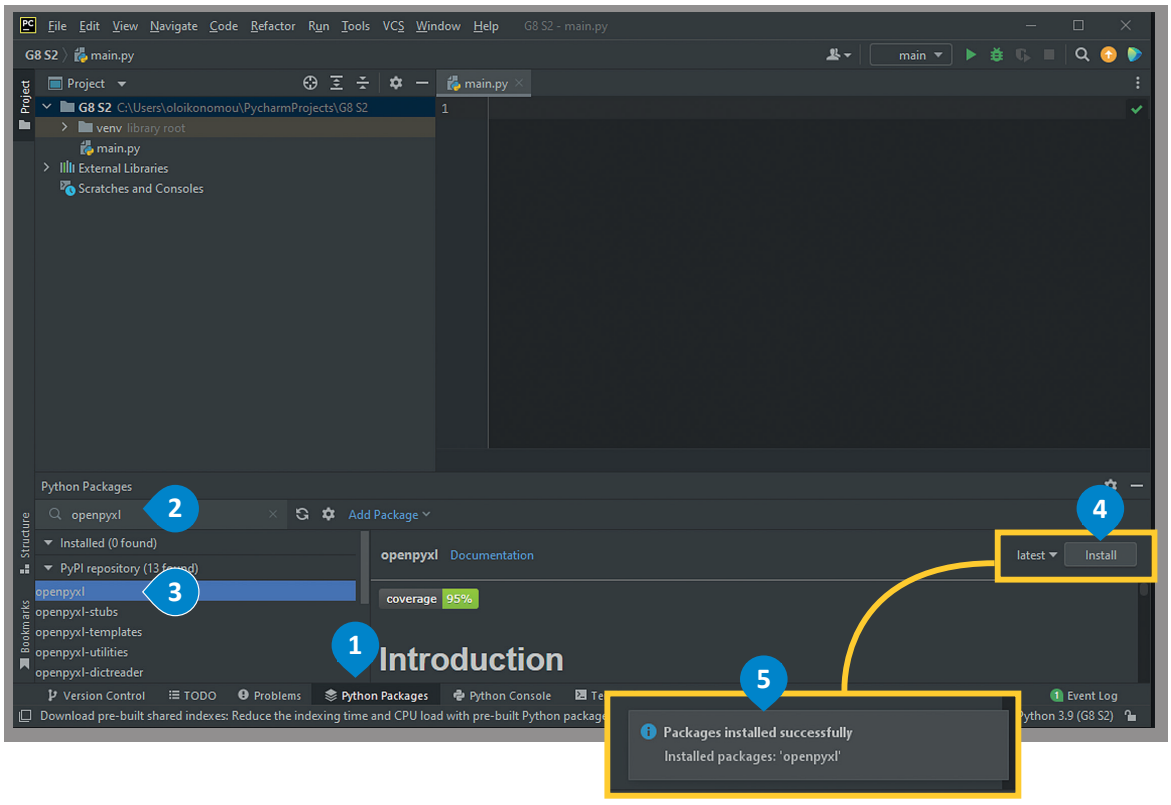
لتثبيت مكتبة أوبين بيكسل (openpyxl):

> في نافذة **PyCharm** (باي تشارم)، اضغط على **Packages** (الحزم). 1

> اكتب **openpyxl** (أوبين بيكسل) في شريط البحث. 2

> اختر **openpyxl** (أوبين بيكسل)، 3 واضغط على **Install** (تثبيت). 4

> ستظهر رسالة تُخبرك بأن التثبيت قد اكتمل. 5



وزارة التعليم
Ministry of Education
2026 - 1448

استيراد مكتبة أوبين بيكسل

لاستخدام مكتبة أوبين بيكسل بعد تثبيتها، تحتاج إلى استيرادها. يمكنك القيام بذلك باستخدام أمر **الاستيراد** (import).

```
# استيراد مكتبة أوبين بيكسل
import openpyxl
```

العمل مع دفاتر العمل

كما تعلم، يمكن أن تحتوي ملفات إكسل على أكثر من ورقة عمل واحدة، ويمكنك إدخال البيانات ومعالجتها في كل ورقة. في مكتبة أوبين بيكسل، يُطلق على ملف إكسل اسم **دفتر العمل** (Workbook)، ويمكنك إما إنشاء ملف إكسل جديد أو تحميل ملف موجود سابقًا.

دالة تحميل دفتر العمل (load_workbook())

للعمل على ملف إكسل موجود ستستخدم دالة **تحميل دفتر العمل** (load_workbook()). وعليك وضع الملف في نفس المجلد مع جزء المقطع البرمجي، أو استخدام المسار المطلق للملف.

سمات أسماء أوراق العمل (sheetnames)

عادَة ما تعمل على ورقة محددة من دفتر العمل، ومن المفيد جدًا معرفة أسماء هذه الأوراق. لرؤية أسماء الأوراق التي يحتوي عليها دفتر العمل، يمكنك استخدام سمة **أسماء أوراق العمل** (sheetnames).

```
import openpyxl

path = "nutrition.xlsx"
wb = openpyxl.load_workbook(path)

# لرؤية أسماء الأوراق في دفتر العمل
sheet_name = wb.sheetnames
print("يحتوي ملف إكسل على أوراق العمل التالية: ",sheet_name)
```

مسار ملف إكسل

تعدّ دالة تحميل دفتر العمل (load_workbook()) وحدة داخل مكتبة أوبين بيكسل (openpyxl)

يحتوي ملف إكسل على الأوراق التالية: ["english", "arabic"]

سمة الصف الأقصى (max_row attribute) و سمة العمود الأقصى (max_column attribute)

عند العمل باستخدام ملف إكسل، فمن الجيد معرفة عدد الصفوف والأعمدة الموجودة في الورقة. للقيام بذلك، يمكنك استخدام سمة **الصف الأقصى** (max_row) و سمة **العمود الأقصى** (max_column) في ورقة العمل.

السمات (Attributes) هي بعض الخصائص المضمنة لنوع خاص من المتغيرات، وتسمى الكائنات.

```
import openpyxl
path = "nutrition.xlsx"
wb = openpyxl.load_workbook(path)
sheet_obj = wb['arabic']

row = sheet_obj.max_row
column = sheet_obj.max_column
print("إجمالي السطور:", row)
print("إجمالي الأعمدة:", column)
```

يُحمّل هذا الأمر الورقة المسماة "arabic" في متغير sheet_obj.

```
إجمالي الصفوف: 12
إجمالي الأعمدة: 12
```

الوصول إلى الخلايا

تحتوي كل خلية في ورقة عمل إكسل على خاصيتين رئيستين، هما موقعها في الملف وقيمته. غالبًا ما ستحتاج إلى الوصول إلى قيمة الخلية من أجل طباعتها على الشاشة، أو تغييرها، أو استخدامها لإجراء عمليات حسابية أخرى مثل العثور على مبلغ وما إلى ذلك.

```
import openpyxl
path = "nutrition.xlsx"
wb = openpyxl.load_workbook(path)
sheet_obj = wb['arabic']
# الوصول إلى الخلايا من ورقة العمل كمفاتيح لدفتر العمل
cell = sheet_obj["A1"]
# لعرض نوع الخلية
print(type(cell))
cell_range = sheet_obj["B1" : "B5"]
print(cell)
```

هذا يعني الانتقال من الخلية "B1" الى الخلية "B5"

يُحمّل هذا الأمر الخلية "A1" في sheet_obj.

```
<class 'openpyxl.cell.cell.Cell'>
<Cell "arabic".A1>
```

اسم الورقة وموقع الخلية

وزارة التعليم
Ministry of Education
2026 - 1448

الوصول إلى قيم الخلايا

لقد تعلمت كيفية الحصول على موقع الخلية. لكن في معظم الأوقات، تحتاج إلى قيمتها. للحصول على قيمة الخلية، تحتاج إلى استدعاء سمة قيمة كائن خلية معين.

في هذا المثال ستستخدم ملف إكسل للتغذية لطباعة عدد **السعرات الحرارية** (kCal) من الحليب. كما رأيت من قبل، يحتوي هذا الملف على 12 صفًا و12 صفًا. للحصول على عدد **السعرات الحرارية** (kCal) من الحليب، عليك أولًا العثور على موقعها، وهو "**C3**" ثم يمكنك استدعاء سمة القيمة في هذه الخلية.

	A	B	C	D	E	F	G	H	I	J	K
1							100 جرام				
2	مكونات	الماء جرام	الطاقة سعر حراري	البروتين جرام	الدهون جرام	الكربوهيدرات جرام	الكالسيوم (Ca) ملليجرام	الفوسفور (P) ملليجرام	الحديد (Fe) ملليجرا	الصوديوم (Na) ملليجرام	البوتاسيوم (K) ملليجرام
3	الحليب	88.1	61	3.27	3.25	4.63	123	101	0	38	150
4	البيض	75.8	143	12.4	8.65	0.96	48	184	1.67	129	132
5	الدجاج	69.9	149	23.9	5.23	0	12	184	0.94	117	239
6	برجر بالجبن	48	261	13.9	11.8	23.9	239	201	2.46	508	211
7	التفاح	83.6	65	0.15	0.16	15.6	6	10	0.02	1	104
8	الكعك	8.85	430	5.79	13.7	69.6	29	111	2.28	314	245
9	مثلجات بالشوكولاتة	55.7	216	3.8	11	28.2	109	107	0.93	76	249
10	حليب بالشوكولاتة	1.5	535	7.65	29.7	59.4	189	208	2.35	79	372
11	الخس	94.7	20	1.24	0.26	3.24	35	30	0.95	0	253
12	الموز	74.9	89	1.09	0.33	22.8	5	22	0.26	1	358

```
import openpyxl
path = "nutrition.xlsx"
wb = openpyxl.load_workbook(path)
sheet_obj = wb['arabic']
position = "C3"
cell = sheet_obj[position]
value = cell.value
print(value)
```

```
61
```

الوصول إلى قيمة أكثر من خلية واحدة

افترض الآن أنك تريد الحصول على جميع الحقائق الغذائية للحليب. يمكنك العثور يدويًا على موقع كل خلية ثم استدعاء سمة قيمتها، ولكن ذلك سيستغرق وقتًا طويلًا ومن الممكن أيضًا أن تكون قد ارتكبت بعض الأخطاء. لذلك، يوجد دالتان لهما وظيفتان مفيدتان في اختيار أكثر من خلية واحدة وهما **iter_rows** و **iter_cols**.

دالة Iter_rows

تُستخدام دالة **Iter_rows** عندما تريد الحصول على جميع القيم لصف أو أكثر في ورقة عمل إكسل.

```
iter_rows (min_row=None, max_row=None, min_col=None, max_col=None)
```

فهرس الصف الأصغر والأكبر

فهرس العمود الأصغر والأكبر

إذا لم تحدد الفهرس، فسيبدأ نطاق البحث من الخلية الأولى"A1".

ستستخدم دالة **iter_rows** الآن للحصول على جميع الحقائق الغذائية للحليب. عليك الحصول على جميع مواقع الخلايا لصف الحليب، وهو "**3D**" ثم استدعاء سمة القيمة.

```
import openpyxl

path = "nutrition.xlsx"
wb = openpyxl.load_workbook(path)
sheet_obj = wb['arabic']

# أنشئ متغير صفوف يحتوي على مواقع حقائق التغذية الخاصة بالحليب
rows = sheet_obj.iter_rows(min_row=3, max_row=3, min_col=3, max_col=11)

# المتداخلة لطباعة قيمة كل خلية rof الآن استخدم حلقات
print("حقائق غذائية عن الحليب:", end=" ")
for row in rows:
    for cell in row:
        print(cell.value, end=" ")
```

```
حقائق غذائية عن الحليب: 88.1 61 3.27 3.25 5.63 123 101 0 38 150
```


وزارة التعليم
Ministry of Education
2026 - 1448

دالة Iter_cols

تُستخدام دالة **Iter_cols** عندما تريد الحصول على جميع القيم لعمود أو أكثر في ورقة عمل إكسل.

```
iter_cols (min_col=None, max_col=None, min_row=None, max_row=None)
```

فهرس العمود الأصغر والأكبر

فهرس الصف الأصغر والأكبر

إذا لم تحدد الفهرس، فسيبدأ نطاق البحث من الخلية الأولى "A1".

ستستخدم **Iter_cols** للحصول على عدد السعرات الحرارية (kCal) لجميع المكونات الموجودة في ورقة العمل. ستحتاج إلى الحصول على مواقع جميع الخلايا ثم استدعاء سمة القيمة.

```
import openpyxl

path = "nutrition.xlsx"
wb = openpyxl.load_workbook(path)
sheet_obj = wb['arabic']

# يحتوي على مواقع حقائق الطاقة لجميع المكونات slOC أنشئ متغير
cols = sheet_obj.iter_cols(min_col=3, max_col=3, min_row=3, max_row=12)

# المتداخلة لطباعة قيمة كل خلية rof الآن استخدم حلقات
print("السعرات الحرارية لجميع المكونات:", end=" ")
for col in cols:
    for cell in col:
        print(cell.value, end=" ")
```

السعرات الحرارية لجميع المكونات: 61 143 149 261 65 430 216 535 20 89

كتابة القيم

لقد تعلمت كيفية الحصول على الخلايا والقيم من ورقة العمل. من الأمور المهمة عند العمل على ملفات إكسل هي كتابة القيم في الخلايا، حيث يمكنك إما إنشاء صفوف وأعمدة جديدة وإضافة قيم في خلايا فارغة، أو يمكنك تغيير قيم الخلايا الموجودة.

يمكنك تغيير قيمة الخلية باستخدام دالة **الخلية** (cell).

```
cell(row=None, column=None)
```

فهرس الصف و العمود

ستعمل على ملف التغذية في إكسل وتغير القيم الموجودة. في هذا الملف، ستُحسب قيم العناصر الغذائية لكل مكون في كل 100 جرام. إذا كنت ترغب في تحويل هذه القيم لإظهارها في كل 30 جرامًا من المكونات، فيجب عليك تغيير كل قيمة وفقًا للصيغة التالية:

new_value = old_value * 30 / 100

عند حفظ التغييرات في ملف إكسل من خلال أوامر بايثون، يجب فتح الملف فقط في بايثون وإلا فلن تحدث التغييرات، وستظهر رسالة خطأ في حال كان ملف الإكسل مفتوحًا في برنامج إكسل في نفس الوقت.

عليك أولًا تغيير قيمة "**B1**" من "100 جرام" إلى "30 جرام".

```
import openpyxl

path = "nutrition.xlsx"
wb = openpyxl.load_workbook(path)
sheet_obj = wb["arabic"]
# غير الخلية B1
sheet_obj.cell(row=1, colu
wb.save("nutrition.xlsx")
```

لحفظ دفتر العمل

الآن إذا فتحت ملف إكسل مرة أخرى، فستتغير قيمة الخلية "B1".

	E	F	G	H	I	J	K
1			30 جرام				
2	مكونات	الماء جرام	الطاقة سعر حراري	البروتين جرام	الدهون جرام	الكربوهيدرات جرام	الكالسيوم (Ca) ملليجرام	الفوسفور (P) ملليجرام	الحديد (Fe) ملليجرام	الصوديوم (Na) ملليجرام	البوتاسيوم (K) ملليجرام
3	الحليب	88.1	61	3.27	3.25	4.63	123	101	0	38	150
4	البيض	75.8	143	12.4	8.65	0.96	48	184	1.67	129	132
5	الدجاج	69.9	149	23.9	5.23	0	12	184	0.94	117	239
6	برجر بالجبن	48	261	13.9	11.8	23.9	239	201	2.46	508	211
7	التفاح	83.6	65	0.15	0.16	15.6	6	10	0.02	1	104
8	الكعك	8.85	430	5.79	13.7	69.6	29	111	2.28	314	245
9	مثلجات بالشوكولاتة	55.7	216	3.8	11	28.2	109	107	0.93	76	249
10	حليب بالشوكولاتة	1.5	535	7.65	29.7	59.4	189	208	2.35	79	372
11	الخس	94.7	20	1.24	0.26	3.24	35	30	0.95	0	253
12	الموز	74.9	89	1.09	0.33	22.8	5	22	0.26	1	358

وزارة التعليم
Ministry of Education
2026 - 1448

لتغيير كل خلية تغذية عليك بتطبيق الصيغة ثم حفظ دفتر العمل. للقيام بذلك، عليك استخدام حلقتين **for** ، واحدة تمر عبر كل صف والأخرى تمر عبر كل عمود.

```
import openpyxl

wb = openpyxl.load_workbook(filename="nutrition.xlsx")

sheet_obj = wb["arabic"]

for row in range (3,13):
    for col in range (2,12):
        old_value = sheet_obj.cell(row=row, colu
        new_value = float(old_value) * 30 / 100
        sheet_obj.cell(row=row, column=col).value = new_value

wb.save("nutrition.xlsx")
```

لأن قيم الخلايا هي من نوع **كائن** (object)، ستحتاج إلى تحويلها إلى **عدد عشري** (float) لإجراء العمليات الحسابية

	A	B	C	D	E	F	G	H	I	J	K
1							30 جرام				
2	مكونات	الماء جرام	الطاقة سعر حراري	البروتين جرام	الدهون جرام	الكربوهيدرات جرام	الكالسيوم (Ca) ملليجرام	الفوسفور (P) ملليجرام	الحديد (Fe) ملليجرا	الصوديوم (Na) ملليجرام	البوتاسيوم (K) ملليجرام
3	الحليب	26,43	18,3	0,981	0,975	1,389	36,9	30,3	0	11,4	45
4	البيض	22,74	42,9	3,72	2,595	0,288	14,4	55,2	1	38,7	39,6
5	الدجاج	20,97	44,7	7,17	1,569	0	3,6	55,2	0,282	35,1	71,7
6	برجر بالجبن	14,4	78,3	4,17	3,54	7,17	71,7	60,3	0,738	152,4	63,3
7	التفاح	25,08	19,5	0,045	0,048	4,68	1,8	3	0,006	0,3	31,2
8	الكعك	2,655	129	1,737	4,11	20,88	8,7	33,3	0,684	94,2	73,5
9	مثلجات بالشوكولاتة	16,71	64,8	1,14	3,3	8,46	32,7	32,1	0,279	22,8	74,7
10	حليب بالشوكولاتة	0,45	160,5	2,295	8,91	17,82	56,7	62,4	0,705	23,7	111,6
11	الخس	28,41	6	0,372	0,078	0,972	10,5	9	0,285	0	75,9
12	الموز	22,47	26,7	0,327	0,099	6,84	1,5	6,6	0,078	0,3	107,4

كما ترى فقد تغيرت جميع القيم الغذائية، واستغرقت هذه المهمة وقتًا أقل بكثير من تغيير كل قيمة يدويًا. هذا النوع من الأتمتة مفيد للغاية خاصة إذا كنت تعمل مع ملفات كبيرة جدًا.

لنطبق معًا

تدريب 1

اكتب ناتج المقطع البرمجي التالي:

```
import openpyxl

wb = openpyxl.load_workbook(filename="nutrition.xlsx")
sheet_obj = wb["arabic"]

rows = sheet_obj.iter_rows(min_row=3, max_row=5, min_col=1, max_col=11)
for row in rows:
    for cell in row:
        print(cell.value, end=" ")
    print(" ")
```

الناتج

وزارة التعليم
Ministry of Education
2026 - 1448

تدريب 2

ما وظيفة المقطع البرمجي التالي؟ وما مخرجاته؟

```
import openpyxl

wb = openpyxl.load_workbook(filename="nutrition.xlsx")
sheet_obj = wb["arabic"]

for row in range (3,13):
    value = float(sheet_obj.cell(row=row, column=5).value)
    if (value < 2):
        print(sheet_obj.cell(row=row, column=1).value)
```

الناتج



تدريب 3

اكتب مقطعًا برمجيًا باستخدام مكتبة أوبين بيكسل (openpyxl)، يعمل على تحميل مجموعة بيانات ملف التغذية و طباعة قيم عناوين الأعمدة.

تدريب 4

اكتب مقطعًا برمجيًا باستخدام مكتبة أوبين بيكسل (openpyxl)، يعمل على تحميل مجموعة بيانات ملف التغذية و طباعة جميع القيم الغذائية الأصغر من 25.

مشروع الوحدة

رابط الدرس الرقمي

www.ien.edu.sa

في هذا المشروع، عليك إنشاء دالة بايثون تصمم نمط الساعة الرملية باستخدام النجوم. سيعطى حجم نمط الساعة الرملية كمدخلات في المقطع البرمجي الرئيس.

يتكون هذا النمط في الواقع من نموذجين. النموذج الأول هو نمط هرم يشير للأسفل، والنموذج الثاني هو نمط هرم يشير للأعلى. أنشئ نمط الساعة الرملية من خلال الجمع بين هذين النموذجين.

1 أنشئ دالة تأخذ رقمًا كإدخال ثم ترسم النمط.

أ استخدم حلقات **for** من أجل رسم نمط هرم يشير للأسفل.

ب استخدم حلقات **for** من أجل رسم نمط هرم يشير للأعلى.

```
*********
*******
*****
***
*
***
*****
*******
*********
```

2 أنشئ مقطعًا برمجيًا بلغة بايثون:

> اطلب من المستخدم إعطاء رقم صحيح (حجم الساعة الرملية).

3 استدعِ الدالة لإنشاء النمط.

4 شغّل المقطع البرمجي وتحقق من صحته وعدم وجود أخطاء.

وزارة التعليم
Ministry of Education
2026 - 1448

في الختام

جدول المهارات

المهارة	درجة الإتقان	
	أتقن	لم يتقن
1. إنشاء مقطع برمجي في بيئة التواصل باي تشارم.		
2. استخدام المعاملات الشرطية.		
3. استخدام المعاملات المنطقية.		
4. استخدام الجملة الشرطية البسيطة.		
5. استخدام الجملة الشرطية **if..else**.		
6. استخدام الجملة الشرطية المتداخلة.		
7. إنشاء مقطع برمجي باستخدام حلقة **for**، وحلقة **while** الشرطية.		
8. استخدام عبارة الإيقاف لإنهاء حلقة.		
9. إنشاء مقطع برمجي باستخدام حلقات متداخلة.		
10. إنشاء أنماط باستخدام الحلقات.		
11. إنشاء دوال جديدة.		
12. استيراد مكتبة إلى جزء مقطع برمجي.		
13. عمل جداول البيانات باستخدام بايثون.		

وزارة التعليم
Ministry of Education
2025 - 1446

المصطلحات

Attributes	سمات	Library	مكتبة
Arguments	الوسائط	local Variables	المتغيرات المحلية
Boolean	القيمة المنطقية	Logical Operator	المعامل المنطقي
Code	المقطع البرمجي	Multiple Decisions	قرارات متعددة
Condition	الشرط	Nested Loops	الحلقات المتداخلة
Conditional Operator	المعامل الشرطي	Nesting	التداخل
Decision	قرار	Nesting Conditions	الشروط المتداخلة
Function	الدالة	Operators	المعاملات
Global Variables	المتغيرات العامة	Outer Loop	الحلقة الخارجية
Infinite Loop	الحلقة اللا نهائية	Parameter	مُعامِل
Inner Loop	الحلقة الداخلية	Pattern	النمط
Indentation	المسافة البادئة	Statement	عبارة

وزارة التعليم
Ministry of Education
2025 - 1446

رابط الدرس الرقمي

www.ien.edu.sa

اختبر نفسك

السؤال الأول

حدد الجملة الصحيحة والجملة الخطأ فيما يلي:	صحيحة	خطأ
1. dwg. .svg .eps .ai هي امتدادات لملفات الفيديو.		
2. في الفيديو **عالي الدقة** (HD - High Definition)، تكون صورة الفيديو أو عدد الإطارات بالثانية التي يتم التقاطها أعلى من فيديو **الدقة القياسية** (SD - Standard Definition).		
3. يستخدم برنامج ترميز VC-1 لمقاطع الفيديو عالية الدقة وبث مقاطع الفيديو على الإنترنت.		
4. يمكن أن تتضمن "الحاوية" ترجمات.		
5. بالنسبة للأصوات البشرية، يمكن استخدام **مُعدَّل البِت** (Bit rate) يتراوح بين 64 و 128 كيلوبت / ثانية.		
6. في برنامج تحرير الفيديو المختصر، يمكنك استخدام **مُرشح البُني الداكن** (Sepia Tone) للحصول على صورة بدون ألوان.		
7. درجات الألوان وتصحيح الألوان هي نفسها.		
8. يستخدم تنسيق صور "GIF" بشكل أساسي لرسومات الإنترنت لدعمها للشفافية.		
9. ملفات صور "JPEG" مناسبة للمنشورات الاحترافية والمنشورات ذات التنسيق الكبير.		
10. تنسيقات الصوت غير المضغوطة الأكثر شيوعًا هي MP3 و WMA.		
11. يعرض اسم الملف نوع الملف الذي هو عليه.		

السؤال الثاني

اختر الإجابة الصحيحة:		
1. في الملفات الصوتية، **مُعدَّل العينة** (Sample rate) الأعلى يعني:	جودة أقل بالإضافة إلى مساحة تخزين أقل مطلوبة.	○
	جودة أعلى بالإضافة إلى مساحة تخزين أكبر.	○
	جودة أعلى بالإضافة إلى مساحة تخزين أقل.	○
2. الجزء الأول من التخطيط السينمائي هو:	مخطط القصة (Storyboard).	○
	جدول التصوير (Découpage).	○
	النص (Script).	○
3. لا يعد مكونًا من مكونات البرنامج النصي:	الأحداث (Events).	○
	جدول التصوير (Découpage).	○
	الحوار (Dialogue).	○
4. لا يعد تنسيقًا للصورة:	TIFF	○
	JPEG	○
	MPEG-4	○

وزارة التعليم
Ministry of Education
2026 - 1448

السؤال الثالث

حدد الجملة الصحيحة والجملة الخطأ فيما يلي:	صحيحة	خطأ
1. تستخدم مخططات المعلومات البيانية لنقل رسالة محددة بسرعة.		
2. أحد الخصائص الرئيسة لمخططات المعلومات البيانية هي التوازن.		
3. الجدول الزمني ليس من أنواع مخططات المعلومات البيانية.		
4. الخطوة الأولى في تصميم مخطط المعلومات البياني هي اختيار موضوع.		
5. لتعديل مظهر أي عنصر حدده ثم استخدم الشريط الجانبي.		
6. لا يمكنك تغيير حجم كل عنصر من تصميمك في كانفا.		
7. يمكنك تصدير مخطط المعلومات البياني كملف PDF.		
8. تكون تصميماتك متاحة في صفحة كانفا الرئيسة.		
9. لا يمكنك إنشاء مخطط معلومات بحجم مخصص في كانفا.		
10. تساعد الخلفية على التركيز على عناصر التصميم الأساسية لمخطط المعلومات البياني.		
11. تساعد الصور في إنشاء اتصال مع النص ويمكن أن توضح المعلومات المقدمة في مخطط المعلومات البياني.		
12. الطباعة من خلال تطبيق كانفا لها تكلفة إضافية.		

السؤال الرابع

حدد الجملة الصحيحة والجملة الخطأ فيما يلي:	صحيحة	خطأ
1. يمكنك استخدام كانفا لإنشاء كتاب إلكتروني.		
2. لا يمكنك تحميل صورك الخاصة في كانفا.		
3. لا يمكنك إنشاء مخطط المعلومات البياني الخاص بك دون استخدام قالب في كانفا.		
4. يمكنك نقل العناصر في كانفا باستخدام طريقة السحب والإفلات.		
5. يمكنك حذف عنصر من تصميمك بالضغط على مفتاح Enter.		
6. لا يمكنك إنشاء حساب في كانفا باستخدام إكس X (تويتر سابقًا).		
7. كانفا مخصص للمصممين فقط.		
8. يمكنك تنزيل تصميم من كانفا بتنسيق exe.		
9. التسجيل لاستخدام كانفا اختياري.		
10. يحفظ كانفا تصميماتك تلقائيًا.		
11. يمكنك وضع عنصر في كانفا من خلال الضغط عليه.		
12. يمكنك استيراد وتحرير ملف PDF في كانفا.		

وزارة التعليم
Ministry of Education
2026 - 1448

السؤال الخامس

في الجدول التالي، يمكنك رؤية قيمة كل متغير أثناء تنفيذ المقطع البرمجي. املأ الفراغ في كل صف من المقطع البرمجي حسب الجدول:

	x	y	z	الشرط
`x=int(input("من فضلك أدخل قيمة المتغير x:"))`	6			
`y=int(input("من فضلك أدخل قيمة المتغير y:"))`		3		
`____=0`			0	
`if x____y:`				True
`____=x____y`			9	
`print (____,____,____)`				
`if x!=____ and y!=____:`				True
`____=x____y`	3			
`y=x____y`		0		
`z=____+y`			3	
`print (____,____,____)`				

```
من فضلك أدخل قيمة المتغير x:
6
من فضلك أدخل قيمة المتغير y:
3
9 3 6
3 0 3
```

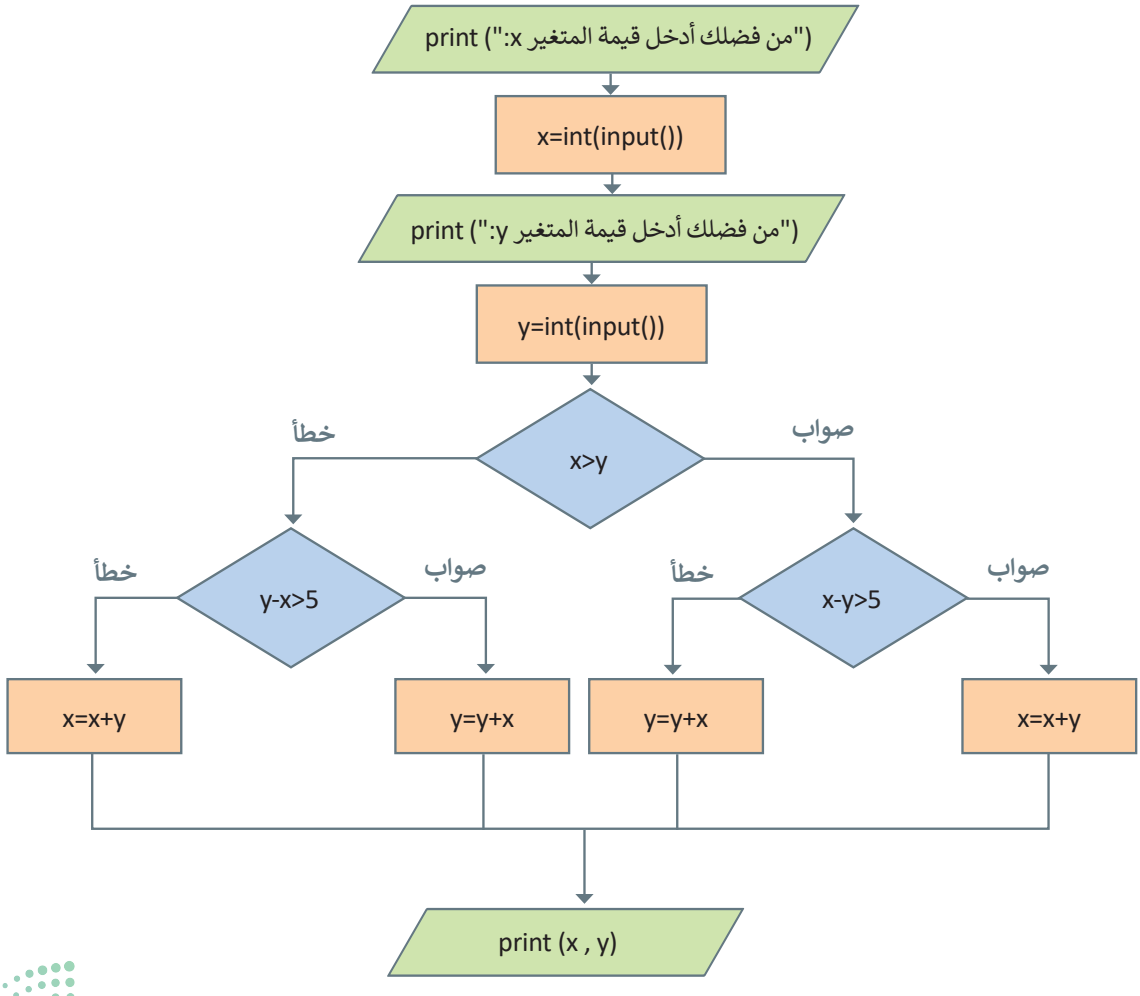
السؤال السادس

اكتب ناتج المخطط الانسيابي التالي

مستخدمًا القيم:

a: x=5, y=12 **ناتج المخطط الانسيابي:** a

b: x=12, y=5 **ناتج المخطط الانسيابي:** b



وزارة التعليم
Ministry of Education
2026 - 1448

السؤال السابع

أدخِل قيم المتغيرات أثناء تنفيذ المقطع البرمجي. افترض أن إدخال المتغير x هو دائما الرقم 15.

```
a = 0
for i in range(1,6):
    x = int(input())
    a = a + x
    if i%5 = 0:
        y = a / 5
        print(a,x,y)
```

i	a	x	y	الشرط	الناتج

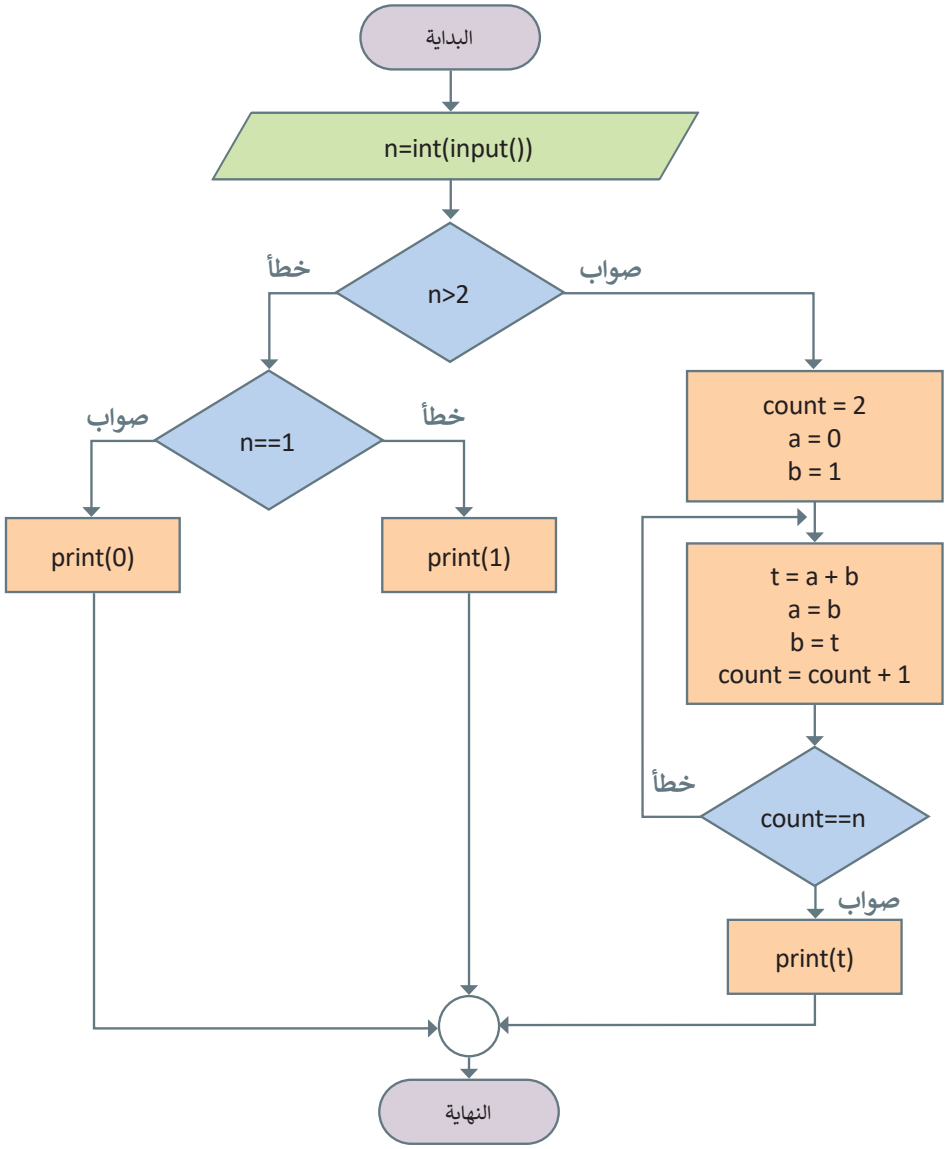
وزارة التعليم
Ministry of Education
2026 - 1448

السؤال الثامن

اكتب ناتج المخطط الانسيابي التالي مستخدمًا القيم:

n=7 ناتج المخطط الانسيابي: a

n=10 ناتج المخطط الانسيابي: b



وزارة التعليم
Ministry of Education
2026 - 1448